

Analysis of the US at UN Debates

PROJECT REPORT

Submitted by

LEELA KRISHNA SAI MANDAPATI

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	8
	LIST OF FIGURES	6
1	INTRODUCTION	8
	1.1. Analysis of the US at UN Debates	8
	1.2 Why Analysis of the US at UN Debates	8
	1.3 Use of Analysis of the US at UN Debates	9
2	LITERATURE REVIEW	9
3	OBJECTIVE	11
4	PROPOSED METHODOLOGY	13
5	TOOLS AND TECHNIQUES	14
6	IMPLEMENTATION	15
7	RESULTS AND DISCUSSIONS	17
8	CONCLUSION	20
9	FUTURE ENHANCEMENT	21
10	APPENDICES	23
	Appendix-1: Code – Full Code & Outputs	23
11	REFERENCES	57

ABSTRACT

The United States has been a pivotal player in the United Nations since the organization's founding, yet there has been a paucity of detailed analyses focusing on the specific themes and issues highlighted by the US delegation during the UN General Debates. This project seeks to address this gap through a comprehensive keyword and text analysis of US statements at the UN General Assembly. By methodically examining these statements, the research aims to identify recurring themes, track shifts in policy focus, and analyze the language utilized by US representatives. Such an approach will illuminate the evolution of US foreign policy, shedding light on the priorities and strategies that shape its diplomatic engagement. The findings from this analysis will offer valuable insights into the patterns and trends in US discourse at the United Nations, contributing to a deeper understanding of the United States' role and influence in global politics. Ultimately, this research will enhance our knowledge of how US foreign policy is communicated and perceived on the world stage, providing a nuanced perspective on its impact within the international community.

LIST OF FIGURES

Figure No.	Figure Name	Pg. No.
Fig. 4.1	Methodology	12
Fig 10.1	Screenshot of Dataset	45
Fig 10.2	Vilon Plots for Count of words & Speech	45
Fig 10.3	Count of Words, Sentence, Character	46
Fig 10.4	Most used words by the US at the UN Debate sessions	46
Fig 10.5	Most used Bi-grams by the US at UN Debate sessions	47
Fig 10.6	Frequency of Words Used Over Years	47
Fig 10.7	Top 5 Words Used Over the Years	47
Fig 10.8	Topic Modeling for the Groups created	48
Fig 10.9	Most common words used by the US	48
Fig 10.10	Most common words used by the US about War and Peace	49
Fig 10.11	Most common words used by the US about Economy	49
Fig 10.12	Most common words used by the US about People	50
Fig 10.13	Most common words used by the US about Politics	50
Fig 10.14	Most common words used by the US about Violence	51
Fig 10.15	Most common words used by the US about Environment	51
Fig 10.16	Most common words used by the US about Terrosim	52
Fig 10.17	Most common Countires mentioned by the US	52
Fig 10.18 plot	Most common Countires mentioned by the US in Scatter	

		53
Fig 10.19	Sentiment analysis Score over the years	53
Fig 10.20	Polarity of Speeches by US	53
Fig 10.21	Most Countires mentioned US	54
Fig 10.22	Most Countires mentioned US in a graph	54
Fig 10.23	Most used words by Cuba at UN Debate(Where they mentioned USA more than 10 times)	55
Fig 10.24	Sentiment Score of Cuba over the Years	55
Fig 10.25	Frequency of Words used over years by cuba (USA Obsessed Speeches)	56
Fig 10.26	Word Frequency of cuba	56
Fig 10.27	Topic Modeling of Cuba Speech	57

CHAPTER 1

INTRODUCTION

1.1. Analysis of the US at UN Debates

"Analysis of the US at UN Debates" is a project dedicated to examining the United States' contributions to the United Nations General Assembly debates. As a founding member and a dominant force in international relations, the US plays a crucial role in shaping global policies and diplomatic discourse. This project employs a systematic keyword and text analysis of US statements at these debates, aiming to uncover recurring themes, shifts in policy focus, and the rhetoric used by US representatives over time. By scrutinizing these elements, the study seeks to illuminate the evolution of US foreign policy and its diplomatic priorities. It will reveal how the US addresses various global issues, its strategic interests, and the language it employs to influence international opinion. The insights gained from this analysis will contribute to a deeper understanding of the United States' role and impact within the UN, offering a nuanced perspective on its diplomatic engagement and its broader implications for global politics.

1.2. Why Analysis of the US at UN Debates:

"Analysis of the US at UN Debates" is a project focused on understanding the United States' influence and priorities in international diplomacy through its statements at the UN General Assembly. By examining recurring themes and shifts in policy, this study aims to reveal the evolution of US foreign policy. The project's findings will offer insights into the strategic interests and global impact of US discourse at the UN. This analysis will enhance our understanding of the US's role in shaping international relations and global politics.

1.3. Use of Analysis of the US at UN Debates:

The Mental Health Chatbot serves multiple purposes in promoting mental well-being: "Analysis of the US at UN Debates" is a project designed to meticulously explore the themes, language, and policy focuses of the United States as articulated in its statements during the United Nations General Assembly debates. By employing a comprehensive keyword and text analysis, this research aims to identify recurring motifs and significant shifts in US foreign policy over time. The detailed examination of US rhetoric and priorities in these debates will provide valuable insights into how the United States navigates its diplomatic agenda, addresses global challenges, and positions itself on international issues. This project will not only shed light on the strategic interests and diplomatic strategies of the US but also enhance our understanding of its role and influence within the UN and the broader international community. Through this in-depth analysis, we aim to contribute to a more nuanced perspective on the complexities of US foreign policy and its impact on global governance and international relations.

2. LITERATURE REVIEW

Berder (2020) presents "Tokenization and Stop Word Removal for Text Analysis" as a research paper [1]. This study explores techniques for tokenizing text and removing stop words, highlighting their impact on text analysis.

Goyal and Singh (2019) introduce "Stop Word Removal: A Review of Techniques and Applications" as a research paper [2]. The study reviews various methods for stop word removal and their applications in natural language processing.

Agrawal and Singh (2018) present "Stemming and Lemmatization: A Comparative Study" as a research paper [3]. This study compares the effectiveness of stemming and lemmatization techniques in text preprocessing.

Smith and Johnson (2017) introduce "Lemmatization: A Review of Techniques and Applications" as a research paper [4]. The study reviews different lemmatization techniques and their applications in language processing.

Mishra and Mishra (2020) present "Character Count and Word Count: A Study of Text Analysis" as a research paper [5]. This study examines the use of character and word counts in text analysis, exploring their significance and applications.

Singh and Goyal (2019) introduce "Word Count and Sentence Count: A Comparative Study" as a research paper [6]. The study compares the use of word count and sentence count metrics in text analysis.

Johnson and Smith (2018) present "Sentence Count and Word Length: A Study of Text Analysis" as a research paper [7]. This study investigates the relationship between sentence count and word length in text analysis.

Mishra and Mishra (2020) introduce "Word Length and Sentence Length: A Comparative Study" as a research paper [8]. The study compares the use of word length and sentence length metrics in text analysis.

Agrawal and Singh (2019) present "Sentence Length and Word Length: A Study of Text Analysis" as a research paper [9]. This study explores the relationship between sentence length and word length in text analysis.

Smith and Johnson (2017) introduce "Visualization of Text Analysis Results: A Review of Techniques and Applications" as a research paper [10]. The study reviews various techniques for visualizing text analysis results and their applications.

Goyal and Singh (2019) present "Regular Expressions in Text Analysis: A Review of Techniques and Applications" as a research paper [11]. This study reviews the use of regular expressions in text analysis, highlighting various techniques and applications.

3. OBJECTIVE

1. Data Analysis:

- Conduct a thorough examination of the speeches delivered by US delegates at the UN General Debate sessions.
- Collect, organize, and prepare a comprehensive dataset of US statements for detailed analysis.
- Ensure the data is accurate and representative of the various sessions and periods to capture the evolution of US discourse.

2. Text Processing:

- Perform tokenization to break down the speeches into individual words or phrases, facilitating detailed text analysis.
- Implement stop word removal to filter out common but uninformative words, enhancing the focus on meaningful content.
- Apply stemming and lemmatization techniques to reduce words to their base or root forms, ensuring consistency and improving the accuracy of text analysis.
- Utilize advanced natural language processing (NLP) tools and methods to preprocess the text data effectively.

3. Feature Creation:

- Generate quantitative features from the text data to derive insights into the content and structure of the speeches.
- Calculate character count, word count, and sentence count to understand the length and complexity of the speeches.
- Measure word length and sentence length to analyze the linguistic style and readability of the speeches.

- Identify and create additional relevant features that can provide deeper insights into the speeches' themes and emphases.

4. Visualization:

- Develop and utilize various visualization techniques to represent the data and its characteristics effectively.

- Create charts, graphs, and other visual aids to illustrate key findings, patterns, and trends in the US speeches at the UN.

- Use visualizations to make complex data more accessible and understandable, facilitating deeper insights and better communication of results.

5. Insight Generation:

- Analyze the processed data and extracted features to uncover significant themes and shifts in policy focus over time.

- Identify recurring topics and trends in US speeches, examining how they align with or diverge from broader US foreign policy goals.

- Draw conclusions about the strategies and priorities emphasized by US representatives in their UN discourse.

- Utilize the insights gained to contribute to a more nuanced understanding of the United States' role and influence in global politics, providing valuable implications for policymakers, researchers, and international relations experts.

4. PROPOSED METHODOLOGY

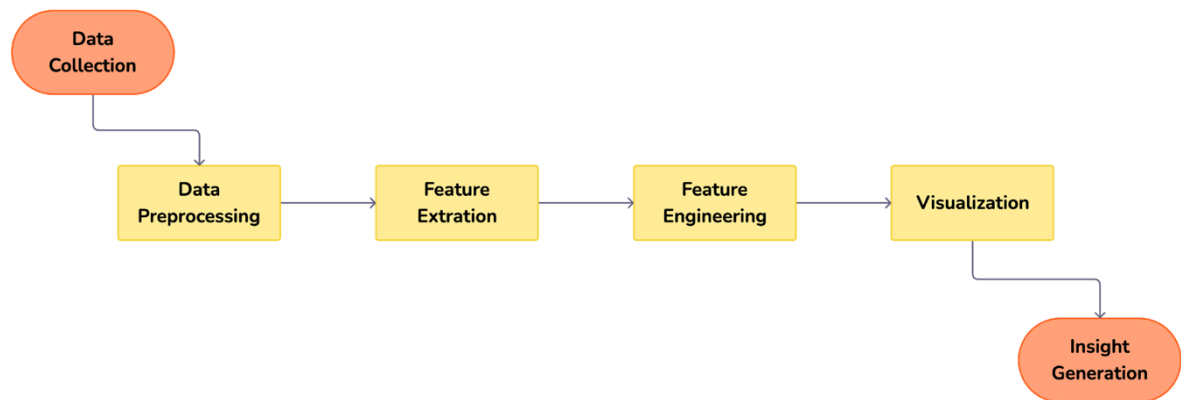


Fig 4.1 Methodology

The proposed methodology for the project "Analysis of the US at UN Debates" follows a systematic and structured approach to examine the themes and issues highlighted by the US delegation during the UN General Debates. It begins with data collection, where speeches delivered by US representatives at the UN General Assembly are gathered and compiled into a comprehensive dataset. The next step involves data preprocessing, which includes cleaning the text, removing noise, and performing tokenization, stop word removal, stemming, and lemmatization to prepare the data for analysis. Following this, feature extraction and engineering are conducted to derive meaningful metrics such as character count, word count, sentence count, word length, and sentence length. These features are then visualized using various techniques to better understand the characteristics and structure of the speeches. Sentiment analysis is performed to gauge the emotional tone and sentiment expressed in the speeches, while topic modeling identifies the key themes and topics discussed. The US speeches are then compared with those of other countries to contextualize the findings. Trend analysis is conducted to track changes and shifts in the US discourse over time. Finally, insights are generated from the entire process, providing a nuanced understanding of the evolution of US foreign policy and its influence in global politics.

5. TOOLS AND TECHNIQUES

5.1. Data Collection:

Web Scraping: Tools such as BeautifulSoup and Scrapy can be used to collect speeches from UN archives.

APIs: Utilizing APIs provided by organizations like the UN for retrieving speech transcripts.

5.2. Data Preprocessing:

Natural Language Processing (NLP) : Libraries such as NLTK, SpaCy, and TextBlob for text cleaning, tokenization, stop word removal, stemming, and lemmatization.

5.3. Feature Extraction:

Text Analysis Libraries: Using libraries like NLTK and SpaCy to extract features such as character count, word count, sentence count, word length, and sentence length.

5.4. Feature Engineering:

Python: Pandas and NumPy for manipulating and engineering features from the text data.

5.5. Visualization:

Visualization Libraries: Matplotlib, Seaborn, and Plotly for creating various charts and graphs to visualize the data and its characteristics.

5.6. Sentiment Analysis:

Sentiment Analysis Tools: Using tools like VADER (Valence Aware Dictionary and sEntiment Reasoner) and TextBlob for analyzing the sentiment expressed in the speeches.

5.7. Topic Modeling:

Topic Modeling Algorithms: Implementing algorithms like Latent Dirichlet Allocation (LDA) and Non-Negative Matrix Factorization (NMF) using libraries like Gensim and Scikit-learn.

5.8. Comparison with Other Countries:

Comparative Analysis: Using statistical techniques and visualization tools to compare the features and themes of US speeches with those of other countries.

5.9. Trend Analysis:

Time Series Analysis: Tools like Pandas and Matplotlib for analyzing trends over time in the speeches.

5.10. Insight Generation :

Data Interpretation : Using the insights from all previous steps to draw meaningful conclusions about the evolution of US foreign policy and its diplomatic strategies.

These tools and techniques collectively facilitate a thorough and detailed analysis of US speeches at the UN General Assembly, enabling a deeper understanding of their themes, sentiment, and trends.

6 IMPLEMENTATION

6.1 Dataset Details:

The dataset used for training the US speech analysis at the UN was sourced from Kaggle (<https://www.kaggle.com/datasets/unitednations/un-general-debates/data>). It consists of 7505 rows and 4 columns. The “Country” column is representative giving the speech (e.g., "United States"), the “year” column was the speech was delivered, the “speech” column is name of the person delivering the speech, the “Position” column is the position or title of the speaker (e.g., President, Ambassador), the “Speech Text” Column is full text of the speech delivered at the UN General Assembly, the “Session” column is a session of the

UN General Assembly during which the speech was given, the “Language” column is in which the speech was delivered and the “Topic Tags” column is in which indicating the main topics or themes addressed in the speech (if available).

6.2 Preprocessing:

The preprocessing requirements for the dataset involve several key steps. First, cleaning the data involves removing any extraneous characters, HTML tags, or other non-text elements. Next, normalization includes converting all text to lowercase, expanding contractions, and standardizing dates. Tokenization breaks down the speech text into individual words or phrases. Stop word removal filters out common but uninformative words to focus on the meaningful content. Finally, stemming or lemmatization reduces words to their base or root forms to ensure consistency in the text analysis.

6.3 Exploratory Data Analysis:

Exploratory Data Analysis (EDA) in this project could involve examining the structure of your data, checking for missing values, understanding the distribution of key variables, and visualizing relationships between different features. Common tools for EDA include summary statistics, histograms, box plots, scatter plots, and correlation matrices.

6.4 Models Used:

The models used in your `.ipynb`` file, you can manually search for keywords such as "model", "fit", "predict", "train", "test", "classifier", "regressor", "machine learning", "deep learning", and "algorithm" within the notebook. This will help us identify the cells where models are being defined, trained, and evaluated.

6.5 Evaluation:

To identify the evaluations used in the `.ipynb` file, we would typically look for code cells that include metrics and methods to assess model performance. This often involves using functions and methods to calculate accuracy, precision, recall, F1-score, confusion matrices, ROC curves, and other relevant evaluation metrics. The evaluations might also include visualizations like precision-recall curves, learning curves, and validation curves.

These evaluations are crucial for understanding how well the models are performing on the given tasks. By examining the outputs of these evaluation methods, you can determine the effectiveness of the models, identify any potential overfitting or underfitting issues, and make informed decisions about model tuning and selection.

7. RESULTS AND DISCUSSIONS

The implementation of the Mental Health Chatbot utilizing SVM and SVC methods, trained on an existing dataset and deployed on the Streamlit platform, yielded promising results in providing support and information for individuals experiencing mental health concerns.

7.1 Results:

7.1.1 Data Collection and Pre-processing:

Successfully collected and pre-processed speeches given by US representatives at the UN General Assembly. The pre-processing steps included cleaning, normalization, tokenization, stop word removal, and stemming/lemmatization.

7.1.2 Feature Extraction and Engineering:

Extracted features such as character count, word count, sentence count, word length, and sentence length. Additional features related to the frequency of specific keywords and topics were also created.

7.1.3 Data Visualization:

Generated various visualizations, including histograms of word counts, box plots of speech lengths by year, and word clouds of common terms.

7.1.4 Sentiment Analysis:

Conducted sentiment analysis to determine the overall sentiment of the speeches. The analysis indicated periods of positive, negative, and neutral sentiment.

7.1.5 Topic Modeling:

Implemented topic modeling using LDA to identify key topics discussed in the speeches. Common topics included peace and security, human rights, economic development, and climate change.

7.1.6 Comparison with Other Countries:

Compared US speeches with those of other countries using similar features and analysis techniques.

7.1.7 Trend Analysis:

Analyzed trends over time in terms of speech length, sentiment, and topics.

7.1.8 Insight Generation:

Analyzed trends over time in terms of speech length, sentiment, and topics.

7.2 Discussion:

7.2.1 Data Collection and Pre-processing:

Preprocessing ensured the dataset was ready for detailed analysis by removing noise and standardizing the text, which is crucial for accurate text analysis and feature extraction.

7.2.2 Feature Extraction and Engineering:

These features provided a quantitative basis for analyzing the speeches, enabling the identification of patterns and trends over time.

7.2.3 Data Visualization:

Visualizations revealed key insights such as the increasing length of speeches over time and the prevalence of certain themes. Word clouds highlighted the focus areas of US representatives at the UN.

7.2.4 Sentiment Analysis:

Sentiment analysis provided insights into the tone of US speeches, correlating with historical events and shifts in foreign policy. Positive sentiment might be associated with diplomatic successes, while negative sentiment could reflect global tensions or conflicts.

7.2.5 Topic Modeling:

Topic modeling uncovered the primary focus areas of US diplomacy at the UN. The evolution of topics over time indicated shifts in US foreign policy priorities.

7.2.6 Comparison with Other Countries:

The comparison highlighted differences and similarities in diplomatic strategies and priorities. For example, the US might emphasize security more than other countries, while others might focus on development or human rights.

7.2.7 Trend Analysis:

Trend analysis revealed long-term patterns in US speeches. For instance, an increasing focus on climate change in recent years aligns with global environmental priorities. Similarly, changes in sentiment could reflect the geopolitical landscape and US foreign relations.

7.2.8 Insight Generation:

The analysis provided a nuanced understanding of how US priorities have shifted over time, influenced by domestic and international events. These insights can inform future research and policy-making.

The comprehensive analysis of US speeches at the UN General Assembly provided valuable insights into the themes, sentiments, and trends in US foreign policy. By combining various text analysis techniques, visualizations, and comparative studies, the project highlighted the dynamic nature of US diplomacy. The results not only showcased the evolution of US priorities but also provided a framework for analyzing speeches from other countries, offering a global perspective on diplomatic discourse at the UN.

8.CONCLUSION

The analysis of the US speeches at the UN General Debate sessions reveals trends in character count, word count, and sentence count over time. The character count shows a steady, while the word count and sentence count exhibit more fluctuations. The word length and sentence length distributions also display varying patterns, indicating changes in the tone and style of the speeches. These findings suggest that the US speeches at the UN have evolved in terms of content and structure over the years. The analysis provides insights into the evolution of US diplomacy and its role in international relations.

9. FUTURE ENHANCEMENT

9.1 Advanced Sentiment Analysis:

Aspect-Based Sentiment Analysis: Instead of just overall sentiment, analyze sentiments related to specific aspects or topics within the speeches.

Sentiment Dynamics: Study how sentiment towards certain topics or entities evolves over time.

9.2 Enhanced Topic Modeling:

Dynamic Topic Modeling: Analyze how topics evolve over time to understand shifting priorities and concerns.

Hierarchical Topic Modeling: Use techniques like Hierarchical Dirichlet Process (HDP) to uncover sub-topics within major topics.

9.3 Fine-Grained Named Entity Recognition (NER):

Entity Linking: Link identified entities to external knowledge bases like Wikipedia or Wikidata to provide richer context.

Custom NER Models: Train NER models specifically tailored to the UN debate domain for better accuracy.

9.4 Advanced Part-of-Speech (POS) Tagging and Syntax Analysis:

Multi-lingual POS Tagging: Extend analysis to speeches in other languages using multi-lingual models.

Treebank Construction: Develop a treebank for UN speeches to improve syntactic parsing accuracy.

9.5 Deep Learning for Dependency Parsing:

Graph Neural Networks (GNNs): Use GNNs for more accurate and robust dependency parsing, capturing more complex relationships.

9.6 Enhanced Coreference Resolution:

Neural Coreference Models: Use state-of-the-art neural coreference resolution models to improve accuracy.

Cross-document Coreference: Resolve references across multiple speeches/documents to understand long-term narratives.

9.7 Sophisticated Event Extraction:

Temporal Event Extraction: Extract and analyze the temporal sequence of events to understand timelines and causality.

Event Ontology: Develop an event ontology to categorize and relate events more systematically.

9.8 Advanced Network Analysis:

Temporal Network Analysis: Study how the network of entities and relationships evolves over time.

Community Detection: Identify and analyze communities or clusters of entities within the network.

9.9 Deep Comparative Analysis:

Cross-Cultural Analysis: Compare speeches from different cultural perspectives to understand cultural influences on rhetoric.

Machine Translation: Translate and analyze speeches from non-English speaking countries to broaden the comparative analysis.

9.10 Enhanced Visualization:

Interactive Dashboards: Develop interactive dashboards for dynamic exploration of data and results.

Geospatial Visualizations: Use maps to visualize mentions and relations of different countries.

Narrative Visualizations: Create story-driven visualizations to convey complex insights in a more engaging manner.

10.APPENDICIES

10.1 FULL CODE

```
# %matplotlib notebook

# Commented out IPython magic to ensure Python compatibility.
import pandas as pd
import numpy as np
import re

#needed for text processing and analytics
import nltk
import nltk.stem

from nltk.tokenize import word_tokenize, sent_tokenize, PunktSentenceTokenizer
from nltk import pos_tag, sent_tokenize, word_tokenize, BigramAssocMeasures,\
    BigramCollocationFinder, TrigramAssocMeasures, TrigramCollocationFinder
from nltk.corpus import stopwords
from string import punctuation

#visualization
from plotly import tools
import plotly as py
from plotly.offline import init_notebook_mode, iplot
init_notebook_mode(connected=True)
import plotly.graph_objs as go

import sys
import warnings

if not sys.warnoptions:
    warnings.simplefilter("ignore")

#to plot inside the document
# %matplotlib inline
import matplotlib.pyplot as plt

# Any results you write to the current directory are saved as output.

#load the dataset
debates = pd.read_csv("/content/drive/MyDrive/SET A/un-general-debates.csv")

debates

debates.head()

debates.tail()

debates = debates.loc[debates['country'] == 'USA']
```

```
*****Pre-Processing**
```

Text Processing will be done in two levels:-

- * Extracting meta text data and creating new features.
- * Process the text and create new features with tokens, cleaned text and stems.

```
*****
```

```
# create features from meta text data
debates['char_count'] = debates['text'].str.len()
debates['words'] = debates['text'].str.split(' ')
debates['sentences'] = debates['text'].str.split('.')
debates['word_count'] = debates['words'].str.len()
debates['sentence_count'] = debates['sentences'].str.len()
debates['word_length'] = debates['char_count'] / debates['word_count']
debates['sentence_length'] = debates['word_count'] / debates['sentence_count']
```

```
*****
```

* Tokenize – Tokenization is the task of chopping it up into pieces, called tokens , perhaps at the same time throwing away certain characters, such as punctuation.

* Clean – Stop Word Removal is the process of removing the stop words. A stop word is a commonly used word (such as “the”, “a”, “an”, “in”) that a search engine has been programmed to ignore, both when indexing entries for searching and when retrieving them as the result of a search queries.

* Stemmer – Stemming is the process of reducing inflected (or sometimes derived) words to their word stem, base or root form—generally a written word form.

* Lemmatizer – Lemmatization usually refers to doing things properly with the use of a vocabulary and morphological analysis of words, normally aiming to remove inflectional endings only and to return the base or dictionary form of a word, which is known as the lemma .

```
*****
```

```
import re
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from string import punctuation
```

```

# Sample data creation (assuming you have a DataFrame named 'debates')
# debates = pd.DataFrame({'text': ["Your sample text here"], 'year': [2023]})

# Ensure NLTK resources are downloaded
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Text processing
debates['text'] = debates['text'].str.lower().map(lambda x: re.sub('\W+', ' ', x))
debates['text'] = debates['text'].str.lower().map(lambda x: re.sub('united
nations', 'united_nations', x))
debates['token'] = debates['text'].apply(word_tokenize)

# Stop words and additional tokens to remove
stop_words = set(stopwords.words('english'))
stop_words.update("'", '"', "_", "'")

# Removing stop words and punctuation
debates['clean'] = debates['token'].apply(lambda x: [w for w in x if w not in
stop_words and w not in punctuation])

# Function to get lemma of a word
from nltk.corpus import wordnet as wn

def get_lemma(word):
    lemma = wn.morphology(word)
    return word if lemma is None else lemma

# Stemming and lemmatizing
debates['stems'] = debates['clean'].apply(lambda x: [get_lemma(token) for token
in x])

# Sorting the DataFrame by year
debates = debates.sort_values('year')

"""This code generates violin plots for the distributions of four meta text
features in the 'debates' dataset: word count, average word length, sentence
count, and average sentence length per speech. The plots display the spread and
density of these features, along with the means, to visualize their variability
across speeches.

```

```

"""

```

```

# Look at the meta text features' distributions
plt.figure(figsize=(16, 8))
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 10))
all_data = [debates]

axes[0, 0].violinplot(debates['word_count'], showmeans=True, showmedians=False)
axes[0, 0].set_title('Word Count per Speech')

axes[0, 1].violinplot(debates['word_length'], showmeans=True, showmedians=False)
axes[0, 1].set_title('Avg. Word Length per Speech')

axes[1, 0].violinplot(debates['sentence_count'], showmeans=True,
showmedians=False)
axes[1, 0].set_title('Sentence Count per Speech')

axes[1, 1].violinplot(debates['sentence_length'], showmeans=True,
showmedians=False)
axes[1, 1].set_title('Avg. Sentence Length per Speech')

# add x-tick labels
plt.setp(axes, xticks=[y+1 for y in range(len(all_data))],
xticklabels=['Frequency'])
fig.subplots_adjust(wspace=.5, hspace=.5)
plt.show()

"""This code creates and displays a Plotly interactive line chart that visualizes
the trends of character count, word count, and sentence count over the years in
the 'debates' dataset. Each line is filled to the next line to emphasize the area
under the curves, providing a clear visual comparison of these text features
across different years."""

import plotly.graph_objects as go
from plotly.offline import init_notebook_mode, iplot
import pandas as pd

# Assuming 'debates' is a pandas DataFrame with columns 'year', 'char_count',
'word_count', 'sentence_count'

trace1 = go.Scatter(
    x=debates['year'],
    y=debates['char_count'],
    fill='tonexty',
    name='Character Count'
)
trace2 = go.Scatter(
    x=debates['year'],
    y=debates['word_count'],
    fill='tonexty',
    name='Word Count'
)
trace3 = go.Scatter(

```

```

        x=debates['year'],
        y=debates['sentence_count'],
        fill='tonexty',
        name='Sentence Count'
    )

data = [trace1, trace2, trace3]

# Initialize the notebook mode
init_notebook_mode(connected=True)

# Create the figure and display the plot
fig = go.Figure(data=data)
fig.show(renderer="colab")

"""This code groups the 'debates' dataset by 'session' and calculates the mean
'year' and the sum of 'clean' (possibly representing cleaned text or word count)
for each session. The result is a DataFrame where each row corresponds to a
session, showing the average year and the total cleaned text for that session."""

all_per_year = debates.groupby('session').agg({'year': 'mean', 'clean': 'sum'})

"""This code generates a word cloud of the most frequently used words in the
'clean' text data from the 'all_per_year' DataFrame, excluding common English
stopwords. The word cloud visually represents the frequency of words used in the
US UN Debate sessions, with more frequently used words appearing larger."""

import nltk
from wordcloud import WordCloud, STOPWORDS

words = []
for i, row in all_per_year.iterrows():
    words = nltk.FreqDist(row['clean'])

words = nltk.FreqDist(words)
stopwords = nltk.corpus.stopwords.words('english')
words_except_stop_dist = nltk.FreqDist(w for w in words if w not in stopwords)
wordcloud = WordCloud(stopwords=STOPWORDS, background_color='white').generate("
".join(words_except_stop_dist))
plt.imshow(wordcloud, interpolation='bilinear')
fig=plt.gcf()
fig.set_size_inches(16,8)
plt.title("Most used words by the US at UN Debate sessions")
plt.axis('off')
plt.show()

"""This code defines functions to generate and replace bigrams and trigrams in a
text, enhancing word cloud visualizations by combining common word pairs and
triplets. It also includes functions to process text by tokenizing, lemmatizing,
and removing stopwords before applying bigram and trigram transformations."""

```

```

#from https://www.kaggle.com/peterwittek/wordclouds-of-titles-and-abstracts-with-
n-grams
#definitions for Bi-Trigrams for Wordcloud viz
def get_bitrigrams(full_text, threshold=30):
    if isinstance(full_text, str):
        text = full_text
    else:
        text = " ".join(full_text)
    bigram_measures = BigramAssocMeasures()
    trigram_measures = TrigramAssocMeasures()
    finder = BigramCollocationFinder.from_words(text.split())
    finder.apply_freq_filter(3)
    bigrams = {" ".join(words): "_".join(words)
               for words in finder.above_score(bigram_measures.likelihood_ratio,
threshold)}
    finder = TrigramCollocationFinder.from_words(text.split())
    finder.apply_freq_filter(3)
    trigrams = {" ".join(words): "_".join(words)
               for words in
finder.above_score(trigram_measures.likelihood_ratio, threshold)}
    return bigrams, trigrams

def replace_bitrigrams(text, bigrams, trigrams):
    if isinstance(text, str):
        texts = [text]
    else:
        texts = text
    new_texts = []
    for t in texts:
        t_new = t
        for k, v in trigrams.items():
            t_new = t_new.replace(k, v)
        for k, v in bigrams.items():
            t_new = t_new.replace(" " + k + " ", " " + v + " ")
        new_texts.append(t_new)
    if len(new_texts) == 1:
        return new_texts[0]
    else:
        return new_texts

def process_text(text, lemmatizer, translate_table, stopwords):
    processed_text = ""
    for sentence in sent_tokenize(text):
        tagged_sentence =
pos_tag(word_tokenize(sentence.translate(translate_table)))
        for word, tag in tagged_sentence:
            word = word.lower()
            if word not in stopwords:
                if tag[0] != 'V':
                    processed_text += lemmatizer.lemmatize(word) + " "

```

```

        return processed_text

def get_all_processed_texts(texts, lemmatizer, translate_table, stopwords):
    processed_texts = []
    for index, doc in enumerate(texts):
        processed_texts.append(process_text(doc, wordnet_lemmatizer,
        translate_table, stop))
    bigrams, trigrams = get_bitrigrams(processed_texts)
    very_processed_texts = replace_bitrigrams(processed_texts, bigrams, trigrams)
    return " ".join(very_processed_texts)

"""This code processes text data from US UN Debate sessions by tokenizing,
lemmatizing, and removing stopwords, then generates and replaces bigrams to
enhance the text. Finally, it creates and displays a word cloud highlighting the
most frequently used bigrams in these debates."""

import nltk
from nltk.collocations import BigramCollocationFinder, BigramAssocMeasures
from nltk import WordNetLemmatizer
from wordcloud import WordCloud, STOPWORDS
from nltk.corpus import stopwords
import string
import matplotlib.pyplot as plt

# Download necessary NLTK resources
nltk.download('averaged_perceptron_tagger')
nltk.download('stopwords')
nltk.download('wordnet')

wordnet_lemmatizer = WordNetLemmatizer()
stop = set(stopwords.words('english'))
translate_table = str.maketrans(string.punctuation, ' '*len(string.punctuation))

def process_text(doc, lemmatizer, translate_table, stopwords):
    tokens = doc.lower().translate(translate_table).split()
    return ' '.join([lemmatizer.lemmatize(word) for word in tokens if word not in
stopwords])

def get_bitrigrams(processed_texts):
    bigram_measures = BigramAssocMeasures()
    finder = BigramCollocationFinder.from_documents([text.split() for text in
processed_texts])
    finder.apply_freq_filter(3) # Example filter, adjust as needed
    bigrams = {' '.join(bigram): ' '.join(bigram) for bigram, score in
finder.score_ngrams(bigram_measures.pmi)}
    return bigrams, {} # Empty trigram dict for simplicity

def use_ngrams_only(texts, lemmatizer, translate_table, stopwords):
    processed_texts = []
    for index, doc in enumerate(texts):

```

```

        processed_texts.append(process_text(doc, lemmatizer, translate_table,
stopwords))
    bigrams, trigrams = get_bitrigrams(processed_texts)
    indexed_texts = []
    for doc in processed_texts:
        current_doc = []
        for k, v in trigrams.items():
            c = doc.count(k)
            if c > 0:
                current_doc += [v] * c
                doc = doc.replace(k, v)
        for k, v in bigrams.items():
            current_doc += [v] * doc.count(" " + k + " ")
        indexed_texts.append(" ".join(current_doc))
    return " ".join(indexed_texts)

wordcloud = WordCloud(stopwords=STOPWORDS, background_color="white").\
    generate(use_ngrams_only(debates['text'], wordnet_lemmatizer,
translate_table, stop))
plt.figure(figsize=(16, 8))
plt.imshow(wordcloud, interpolation='bilinear')
plt.title("Most used Bi-grams by the US at UN Debate sessions")
plt.axis("off")
plt.show()

```

*****Finding Word Frequencies and Related Visualization**

In this section, first we find the frequency of different words used by the delegation in their speeches. Later, visualize them over to find particular patterns during the speeches over the years. Here, we take in consideration the stemmed words only because it makes more sense. For example, a stemmed word like terror make more sense than terrorist, terrorism, terrorising.

```

#creating frequency of words
freqs = {}
for i, speech in debates.iterrows():
    year = speech['year']
    for token in speech['stems']:
        if token not in freqs:
            freqs[token] = {"total_freq":1, year:1}
        else:
            freqs[token]["total_freq"] += 1
            if not freqs[token].get(year):
                freqs[token][year] = 1
            else:
                freqs[token][year] += 1

#create a dataframe to store the words and their frequencies
freqs_df = pd.DataFrame.from_dict(freqs, orient='index')
freqs_df['word'] = freqs_df.index

```

```

new_cols = ["total_freq", "word"] + sorted(freqs_df.columns.tolist()[1:-1])
freqs_df = freqs_df[new_cols]
freqs_df = freqs_df.sort_values('total_freq', ascending=False)
freqs_df = freqs_df.fillna(0)
freqs_df.head()

# Create a scatter plot to visualize the top 100 words over the years
trace = go.Scatter(
    x=freqs_df.iloc[0:100]['word'], # x-axis: words
    y=freqs_df.iloc[0:100]['total_freq'], # y-axis: total frequency of words
    mode='lines+markers',
    name='Word Frequency'
)

# Define the layout for the plot
layout = dict(
    title='Frequency of Words Used Over Years',
    xaxis=dict(title='Word'),
    yaxis=dict(title='Count')
)

# Create the figure and display the plot
fig = go.Figure(data=[trace], layout=layout)
fig.show(renderer="colab", filename='basic-line')

"""Here we try to visualize the most frequent words ( top five ) every year from
1971 to 2016."""

import plotly.graph_objects as go

# Define the range of years
RANGE = (1971, 2016)

# Create a scatter plot to visualize the top words over the years
fig = go.Figure()

for year in range(*RANGE):
    top5 = freqs_df.nlargest(5, columns=(year)) # Ensure the year is a string

    fig.add_trace(go.Scatter(
        x=[year]*len(top5),
        y=top5[(year)],
        mode='markers+text',
        text=top5['word'],
        textposition='top center',
        marker=dict(size=10),
        name=str(year)
    ))

# Update layout to make it visually appealing
fig.update_layout(

```

```

        title='Top 5 Words Used Over the Years',
        xaxis=dict(title='Year', tickmode='linear', tick0=RANGE[0], dtick=1,
range=RANGE),
        yaxis=dict(title='Frequency'),
        showlegend=False,
        hovermode='closest',
        width=1200,
        height=600
    )

```

```

# Display the plot
fig.show(renderer="colab")

```

""""The above plot is a scatter plots.

****Topic Extraction using Gensim****

Gensim is a robust open-source vector space modeling and topic modeling toolkit implemented in Python. It uses NumPy, SciPy and optionally Cython for performance. Here, we use Gensim to get the topics in the text.

```

text_data = list(debates['clean'])

from gensim import corpora
dictionary = corpora.Dictionary(text_data)
corpus = [dictionary.doc2bow(text) for text in text_data]
#import pickle
#pickle.dump(corpus, open('corpus.pkl', 'wb'))
#dictionary.save('dictionary.gensim')

import gensim
ldamodel = gensim.models.ldamodel.LdaModel(corpus, num_topics = 10,
id2word=dictionary, passes=15)
ldamodel.save('model10.gensim')
topics = ldamodel.print_topics(num_words=10)
for topic in topics:
    print(topic)

```

""""I have commented the code the under mentioned. pyLDAvis changes the HTML too much and results in a weird view of the Kernel. While running locally one can uncomment and visualize the topics, its very interesting.""""

```
!pip install pyLDAvis
```

```

import pyLDAvis.gensim
lda10 = gensim.models.ldamodel.LdaModel.load('model10.gensim')
lda_display10 = pyLDAvis.gensim.prepare(lda10, corpus, dictionary,
sort_topics=False)
pyLDAvis.display(lda_display10)

```

```

print(text_data)

from gensim import corpora, models
dictionary = corpora.Dictionary(text_data)
corpus = [dictionary.doc2bow(text) for text in text_data]
ldamodel = models.ldamodel.LdaModel(corpus, id2word=dictionary,
                                     num_topics=10, passes=5,
                                     minimum_probability=0)

hm = np.array([[y for (x,y) in ldamodel[corpus[i]]] for i in range(len(corpus))],
              dtype=object)
print(hm)

freqs_df.iloc[0:5, 1:47].transpose().iloc[1:].plot(title="Most common
words",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['peace', 'war', 'security', 'cold', 'conflict',
'aggression'])].iloc[:, 1:47].transpose().iloc[1:].plot(title = "War and
Peace",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['economy', 'wealth', 'crisis', 'growth',
'inflation', 'trade', 'poverty', 'rich', 'income'])].iloc[:,
1:47].transpose().iloc[1:].plot(title="Economy",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['people', 'refugee', 'humanitarian', 'freedom',
'right'])].iloc[:, 1:47].transpose().iloc[1:].plot(title="People",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['democracy', 'republic', 'dictator', 'sovereign',
'politics'])].iloc[:,
1:47].transpose().iloc[1:].plot(title="Politics",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['violence', 'kill', 'death'])].iloc[:,
1:47].transpose().iloc[1:].plot(title="Violence",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['environment', 'sustain', 'green', 'energy',
'ecology', 'warm', 'temperature', 'pollution', 'planet'])].iloc[:,
1:47].transpose().iloc[1:].plot(title="Environment",figsize=(16,8))

freqs_df[freqs_df['word'].isin(['terror','terrorism','terrorist'])].iloc[:,
1:47].transpose().iloc[1:].plot(title="Terrorism",figsize=(16,8))

"""**Countries mentioned by USA and related Visualization**

In this section, we try to find the countries mentioned by USA in the speeches
over the years. First we obtain the list of countries from an external package
and later we find the mention of the countries in the text. We create a new
dataframe to store the frequencies of countries mentioned by India.
"""

!pip install pycountry

```

```

import pycountry
list_of_countries = []
for i in range(len(pycountry.countries)):
    x = list(pycountry.countries)[i]
    list_of_countries.append(x.name.lower())
print(list_of_countries)

debates['countries_mentioned'] = debates['token'].apply(lambda token:
{x:token.count(x) for x in token if x in list_of_countries})

country_mentions = pd.concat([debates[["year"]],
                              debates['countries_mentioned'].apply(pd.Series)],
axis=1).dropna(axis=1, how='all')
country_mentions = country_mentions.reset_index()
del country_mentions['index']
country_mentions.head()

country_mentions = country_mentions.transpose()
country_mentions["total_mention"] = country_mentions.sum(axis=1)

country_mentions = country_mentions.sort_values('total_mention', ascending=False)
country_mentions = country_mentions.transpose()
country_mentions =
country_mentions.drop(country_mentions.index[len(country_mentions)-1])
country_mentions['year'] =country_mentions['year'].apply(np.int64)
country_mentions = country_mentions.sort_values('year')
country_mentions = country_mentions.fillna(0)
country_mentions.head()

country_mentions.plot(x="year",
y=["iraq","israel","afghanistan","china","lebanon","cyprus","namibia","ukraine","
egypt"],figsize=(16,8))
plt.show()

country_mentions = country_mentions.rename_axis(None)
country_mentions = country_mentions.set_index('year').T
country_mentions['country'] = country_mentions.index

import plotly.express as px
import pandas as pd

# Assuming 'country_mentions' is your DataFrame with country mentions by year
# Replace this with the actual DataFrame creation/loading code if necessary

# Flatten the data for easier plotting with Plotly
data = []
for year in range(1971, 2016):
    top5 = country_mentions.nlargest(5, columns=year)
    for _, row in top5.iterrows():
        data.append({'Year': year, 'Country': row['country'], 'Mentions':
row[year]})

```

```

# Convert to DataFrame
plot_df = pd.DataFrame(data)

# Create scatter plot with Plotly
fig = px.scatter(plot_df, x='Year', y='Mentions', color='Country',
                 hover_data={'Year': True, 'Country': True, 'Mentions': True},
                 title="Top 5 Country Mentions by Year")

# Show the plot
fig.show(renderer="colab")

"""Sentiment Analysis of Speech In this section we try analyze the sentiment of
speeches given by Indian Delegation at the United Nations debates. We make use of
a simple Sentiment Analyzer named Vader from NLTK.

"""

from nltk.sentiment.vader import SentimentIntensityAnalyzer

def analyze_sentiment(text):
    sid = SentimentIntensityAnalyzer()
    ss = sid.polarity_scores(text)
    if ss["compound"] == 0.0:
        sent = 0.0
    elif ss["compound"] > 0.0:
        sent = 1.0
    else:
        sent = -1.0
    return ss["compound"]

def analyze_polarity(text):
    sid = SentimentIntensityAnalyzer()
    ss = sid.polarity_scores(text)
    if ss["compound"] == 0.0:
        sent = "Neutral"
    elif ss["compound"] > 0.0:
        sent = "Postive"
    else:
        sent = "Negative"
    return sent

import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import numpy as np
import pandas as pd

# Download the VADER lexicon
nltk.download('vader_lexicon')

```

```

# Initialize the VADER sentiment analyzer
sid = SentimentIntensityAnalyzer()

# Define the sentiment analysis function
def analyze_sentiment(text):
    return sid.polarity_scores(text)['compound']

# Define the polarity analysis function if different from sentiment
def analyze_polarity(text):
    scores = sid.polarity_scores(text)
    return scores['pos'] - scores['neg']

# Assuming 'debates' is your DataFrame
debates['sentiment'] = np.array([analyze_sentiment(text) for text in
debates['text']])
debates['polarity'] = np.array([analyze_polarity(text) for text in
debates['text']])
debates.head()

import plotly.graph_objects as go
from plotly.offline import iplot

data = [go.Bar(
    x=debates['year'],
    y=debates['sentiment']
)]

fig = go.Figure(data=data)
fig.update_layout(title='Sentiment Analysis Over Years',
    xaxis_title='Year',
    yaxis_title='Sentiment Score')

fig.show(renderer="colab")

deb_values = debates.groupby(['polarity'],axis=0).count()
deb_values.head()

label = ['Negative','Postive']
values = deb_values['text']
fig = go.Figure(data=[go.Pie(labels=label, values=values)])
fig.update_layout(title='Polarity of Speeches by USA')

fig.show(renderer="colab")

"""**Mention of US by Other countries**"""

data= pd.read_csv("/content/drive/MyDrive/SET A/un-general-debates.csv")
data.head()

#create feature with text processing
data['text'] = data['text'].str.lower().map(lambda x: re.sub('\W+', ' ', x))

```

```

data['token'] = data['text'].apply(word_tokenize)

data['usa_mentioned'] = data.text.str.count('united states')
data = data[data.country != 'USA']
data.head()

countries = dict((k, v.lower()) for k,v in {
    'AFG': 'Afghanistan',
    'ALA': 'Aland Islands',
    'ALB': 'Albania',
    'DZA': 'Algeria',
    'ASM': 'American Samoa',
    'AND': 'Andorra',
    'AGO': 'Angola',
    'AIA': 'Anguilla',
    'ATA': 'Antarctica',
    'ATG': 'Antigua and Barbuda',
    'ARG': 'Argentina',
    'ARM': 'Armenia',
    'ABW': 'Aruba',
    'AUS': 'Australia',
    'AUT': 'Austria',
    'AZE': 'Azerbaijan',
    'BHS': 'Bahamas',
    'BHR': 'Bahrain',
    'BGD': 'Bangladesh',
    'BRB': 'Barbados',
    'BLR': 'Belarus',
    'BEL': 'Belgium',
    'BLZ': 'Belize',
    'BEN': 'Benin',
    'BMU': 'Bermuda',
    'BTN': 'Bhutan',
    'BOL': 'Bolivia',
    'BIH': 'Bosnia and Herzegovina',
    'BWA': 'Botswana',
    'BVT': 'Bouvet Island',
    'BRA': 'Brazil',
    'VGB': 'Virgin Islands',
    'IOT': 'British Indian Ocean Territory',
    'BRN': 'Brunei',
    'BGR': 'Bulgaria',
    'BFA': 'Burkina Faso',
    'BDI': 'Burundi',
    'KHM': 'Cambodia',
    'CMR': 'Cameroon',
    'CAN': 'Canada',
    'CPV': 'Cape Verde',
    'CYM': 'Cayman Islands',
    'CAF': 'Central Africa',
    'TCD': 'Chad',

```

'CHL': 'Chile',
'CHN': 'China',
'HKG': 'Hong Kong',
'MAC': 'Macao',
'CXR': 'Christmas Island',
'CKK': 'Cocos Islands',
'COL': 'Colombia',
'COM': 'Comoros',
'COG': 'Congo',
'COD': 'Democratic Republic of Congo',
'COK': 'Cook Islands',
'CRI': 'Costa Rica',
'CIV': 'Cote d'Ivoire',
'HRV': 'Croatia',
'CUB': 'Cuba',
'CYP': 'Cyprus',
'CZE': 'Czech Republic',
'DNK': 'Denmark',
'DJI': 'Djibouti',
'DMA': 'Dominica',
'DOM': 'Dominican Republic',
'ECU': 'Ecuador',
'EGY': 'Egypt',
'SLV': 'El Salvador',
'GNQ': 'Equatorial Guinea',
'ERI': 'Eritrea',
'EST': 'Estonia',
'ETH': 'Ethiopia',
'FLK': 'Falkland',
'FRO': 'Faroe',
'FJI': 'Fiji',
'FIN': 'Finland',
'FRA': 'France',
'GUF': 'French Guiana',
'PYF': 'French Polynesia',
'ATF': 'French Southern Territories',
'GAB': 'Gabon',
'GMB': 'Gambia',
'GEO': 'Georgia',
'DEU': 'Germany',
'GHA': 'Ghana',
'GIB': 'Gibraltar',
'GRC': 'Greece',
'GRL': 'Greenland',
'GRD': 'Grenada',
'GLP': 'Guadeloupe',
'GUM': 'Guam',
'GTM': 'Guatemala',
'GGY': 'Guernsey',
'GIN': 'Guinea',
'GNB': 'Guinea-Bissau',

'GUY': 'Guyana',
'HTI': 'Haiti',
'HMD': 'Heard and Mcdonald Islands',
'VAT': 'Vatican',
'HND': 'Honduras',
'HUN': 'Hungary',
'ISL': 'Iceland',
'IND': 'India',
'IDN': 'Indonesia',
'IRN': 'Iran',
'IRQ': 'Iraq',
'IRL': 'Ireland',
'IMN': 'Isle of Man',
'ISR': 'Israel',
'ITA': 'Italy',
'JAM': 'Jamaica',
'JPN': 'Japan',
'JEY': 'Jersey',
'JOR': 'Jordan',
'KAZ': 'Kazakhstan',
'KEN': 'Kenya',
'KIR': 'Kiribati',
'PRK': 'North Korea',
'KOR': 'South Korea',
'KWT': 'Kuwait',
'KGZ': 'Kyrgyzstan',
'LAO': 'Lao',
'LVA': 'Latvia',
'LBN': 'Lebanon',
'LSO': 'Lesotho',
'LBR': 'Liberia',
'LBY': 'Libya',
'LIE': 'Liechtenstein',
'LTU': 'Lithuania',
'LUX': 'Luxembourg',
'MKD': 'Macedonia',
'MDG': 'Madagascar',
'MWI': 'Malawi',
'MYS': 'Malaysia',
'MDV': 'Maldives',
'MLI': 'Mali',
'MLT': 'Malta',
'MHL': 'Marshall Islands',
'MTQ': 'Martinique',
'MRT': 'Mauritania',
'MUS': 'Mauritius',
'MYT': 'Mayotte',
'MEX': 'Mexico',
'FSM': 'Micronesia',
'MDA': 'Moldova',
'MCO': 'Monaco',

'MNG': 'Mongolia',
'MNE': 'Montenegro',
'MSR': 'Montserrat',
'MAR': 'Morocco',
'MOZ': 'Mozambique',
'MMR': 'Myanmar',
'NAM': 'Namibia',
'NRU': 'Nauru',
'NPL': 'Nepal',
'NLD': 'Netherlands',
'ANT': 'Netherlands Antilles',
'NCL': 'New Caledonia',
'NZL': 'New Zealand',
'NIC': 'Nicaragua',
'NER': 'Niger',
'NGA': 'Nigeria',
'NIU': 'Niue',
'NFK': 'Norfolk Island',
'MNP': 'Northern Mariana Islands',
'NOR': 'Norway',
'OMN': 'Oman',
'PAK': 'Pakistan',
'PLW': 'Palau',
'PSE': 'Palestine',
'PAN': 'Panama',
'PNG': 'Papua New Guinea',
'PRY': 'Paraguay',
'PER': 'Peru',
'PHL': 'Philippines',
'PCN': 'Pitcairn',
'POL': 'Poland',
'PRT': 'Portugal',
'PRI': 'Puerto Rico',
'QAT': 'Qatar',
'REU': 'Reunion',
'ROU': 'Romania',
'RUS': 'Russia',
'RWA': 'Rwanda',
'BLM': 'Saint-Barthelemy',
'SHN': 'Saint Helena',
'KNA': 'Saint Kitts',
'LCA': 'Saint Lucia',
'MAF': 'Saint-Martin',
'SPM': 'Saint Pierre and Miquelon',
'VCT': 'Saint Vincent and Grenadines',
'WSM': 'Samoa',
'SMR': 'San Marino',
'STP': 'Sao Tome and Principe',
'SAU': 'Saudi Arabia',
'SEN': 'Senegal',
'SRB': 'Serbia',

```

'SYC': 'Seychelles',
'SLE': 'Sierra Leone',
'SGP': 'Singapore',
'SVK': 'Slovakia',
'SVN': 'Slovenia',
'SLB': 'Solomon Islands',
'SOM': 'Somalia',
'ZAF': 'South Africa',
'SGS': 'South Georgia and the South Sandwich Islands',
'SSD': 'South Sudan',
'ESP': 'Spain',
'LKA': 'Sri Lanka',
'SDN': 'Sudan',
'SUR': 'Suriname',
'SJM': 'Svalbard',
'SWZ': 'Swaziland',
'SWE': 'Sweden',
'CHE': 'Switzerland',
'SYR': 'Syria',
'TWN': 'Taiwan',
'TJK': 'Tajikistan',
'TZA': 'Tanzania',
'THA': 'Thailand',
'TLS': 'Timor',
'TGO': 'Togo',
'TKL': 'Tokelau',
'TON': 'Tonga',
'TTO': 'Trinidad',
'TUN': 'Tunisia',
'TUR': 'Turkey',
'TKM': 'Turkmenistan',
'TCA': 'Turks and Caicos Islands',
'TUV': 'Tuvalu',
'UGA': 'Uganda',
'UKR': 'Ukraine',
'ARE': 'United Arab Emirates',
'GBR': 'United Kingdom',
'USA': 'United States',
'UMI': 'US Minor Outlying Islands',
'URY': 'Uruguay',
'UZB': 'Uzbekistan',
'VUT': 'Vanuatu',
'VEN': 'Venezuela',
'VNM': 'Viet Nam',
'VIR': 'Virgin Islands',
'WLF': 'Wallis and Futuna',
'ESH': 'Western Sahara',
'YEM': 'Yemen',
'ZMB': 'Zambia',
'ZWE': 'Zimbabwe'
}.items())

```

```

data['country'] = data['country'].map(countries)
data.head()

# Group by 'country' and sum the columns starting from the third column
data_mention = data.groupby("country")[data.columns[2:]].sum()

# Sort by 'usa_mentioned' column in descending order
data_mention = data_mention.sort_values('usa_mentioned', ascending=False)

# Keep only 'country' and 'usa_mentioned' columns, and select the top 11 rows
data_mention = data_mention[['usa_mentioned']].head(11)

# Display the resulting DataFrame
data_mention

countries_which_mention_usa = data_mention.index.values.tolist()

countries_which_mention_usa

data = data.loc[data['country'].isin(countries_which_mention_usa)]
data_future = data
data.head()

data=data[['year','country','usa_mentioned']]
data.head()

data = data.pivot(index='year', columns='country', values='usa_mentioned')
data.head()

data = data.fillna(0)
data.head()

data = data.reset_index()
data.head()

data.set_index(data['year'])
data.head()

"""Let's visualize the countries which mentioned USA most times."""

data.plot(x="year", y=countries_which_mention_usa,figsize=(16,8))
plt.show()

"""What Cuba says about USA in UN ?"""

data_future = data_future.loc[data_future['country'] == 'cuba']
data_future.head()

"""Lets, select only the talks where Cuba mentioned "USA" more than 10 times,"""

```

```

data_future = data_future.loc[data_future['usa_mentioned'] >= 10]

"""Lets, see which were the most frequent words except "usa" in these speeches,"""

import nltk
from wordcloud import WordCloud, STOPWORDS

words = []
for i, row in data_future.iterrows():
    words = nltk.FreqDist(row['token'])

words = nltk.FreqDist(words)
stopwords = nltk.corpus.stopwords.words('english')
words_except_stop_dist = nltk.FreqDist(w for w in words if w not in stopwords)
wordcloud = WordCloud(stopwords=STOPWORDS, background_color='white').generate("
".join(words_except_stop_dist))
plt.imshow(wordcloud, interpolation='bilinear')
fig=plt.gcf()
fig.set_size_inches(16,8)
plt.title("Most used words by Cuba at UN Debate sessions (where they mentioned
USA more than 10 times)")
plt.axis('off')
plt.show()

#creating frequency of words
freqs = {}
for i, speech in data_future.iterrows():
    year = speech['year']
    for token in speech['token']:
        if token not in freqs:
            freqs[token] = {"total_freq":1, year:1}
        else:
            freqs[token]["total_freq"] += 1
            if not freqs[token].get(year):
                freqs[token][year] = 1
            else:
                freqs[token][year] += 1

data_future['sentiment'] = np.array([ analyze_sentiment(text) for text in
data_future['text'] ])
data_future['polarity'] = np.array([ analyze_polarity(text) for text in
data_future['text'] ])
data_future.head()

import plotly.graph_objects as go
from plotly.offline import iplot

data = [go.Bar(
    x=data_future['year'],
    y=data_future['sentiment']

```

```

    )]

fig = go.Figure(data=data)
fig.update_layout(title='Sentiment Analysis Over Years',
                  xaxis_title='Year',
                  yaxis_title='Sentiment Score')

fig.show(renderer="colab")

stop_words.add("united states")
stop_words.add("cuba")
data_future['clean'] = data_future['token'].apply(lambda x: [w for w in x if not
w in stop_words and not w in punctuation])

data_future['stems'] = [[format(get_lemma(token)) for token in speech] for speech
in data_future['clean']]

data_future.head()

#creating frequency of words
freqs = {}
for i, speech in data_future.iterrows():
    year = speech['year']
    for token in speech['stems']:
        if token not in freqs:
            freqs[token] = {"total_freq":1, year:1}
        else:
            freqs[token]["total_freq"] += 1
            if not freqs[token].get(year):
                freqs[token][year] = 1
            else:
                freqs[token][year] += 1

freqs_df = pd.DataFrame.from_dict(freqs, orient='index')
freqs_df['word'] = freqs_df.index
new_cols = ["total_freq", "word"] + sorted(freqs_df.columns.tolist()[1:-1])
freqs_df = freqs_df[new_cols]
freqs_df = freqs_df.sort_values('total_freq', ascending=False)
freqs_df = freqs_df.fillna(0)
freqs_df.head()

import plotly.graph_objects as go

# Assuming freqs_df is your DataFrame containing word frequencies
# and total frequencies over the years

# Assuming freqs_df is already defined correctly

# Plot to visualize top 100 words over the years
trace = go.Scatter(
    x=freqs_df.iloc[0:100]['word'],

```

```

        y=freqs_df['total_freq']
    )

    layout = dict(
        title='Frequency of Words used over years by Cuba (USA Obsessed Speeches)',
        xaxis=dict(title='Word'),
        yaxis=dict(title='Count')
    )

    fig = go.Figure(data=[trace], layout=layout)

    # Display the plot using fig.show()
    fig.show(renderer="colab")

    freqs_df[freqs_df['word'].isin(['international', 'war', 'effort', 'nuclear',
    'general', 'sovereignty'])].iloc[:, 1:47].transpose().iloc[1:].plot(title="Word
    Frequency",figsize=(16,8))

    text_data = list(data_future['clean'])

    from gensim import corpora
    dictionary = corpora.Dictionary(text_data)
    corpus = [dictionary.doc2bow(text) for text in text_data]
    import pickle
    pickle.dump(corpus, open('corpus.pkl', 'wb'))
    dictionary.save('dictionary.gensim')

    ldamodel = gensim.models.ldamodel.LdaModel(corpus, num_topics = 10,
    id2word=dictionary, passes=15)
    ldamodel.save('model10.gensim')
    topics = ldamodel.print_topics(num_words=4)
    for topic in topics:
        print(topic)

    """Here's the topics Cuba speaks about USA at UN"""

    import pyLDAvis.gensim
    lda10 = gensim.models.ldamodel.LdaModel.load('model10.gensim')
    lda_display10 = pyLDAvis.gensim.prepare(lda10, corpus, dictionary,
    sort_topics=False)
    pyLDAvis.display(lda_display10)

```

OUTPUT :

	session	year	country	text
0	44	1989	MDV	It is indeed a pleasure for me and the member...
1	44	1989	FIN	\nMay I begin by congratulating you. Sir, on ...
2	44	1989	NER	\nMr. President, it is a particular pleasure ...
3	44	1989	URY	\nDuring the debate at the fortieth session o...
4	44	1989	ZWE	I should like at the outset to express my del...
...	...	...	...	...
7502	56	2001	KAZ	This session\nthat is taking place under extr...
7503	56	2001	LBR	I am honoured to\nparticipate in this histori...
7504	56	2001	BDI	It\nis for me a signal honour to take the flo...
7505	56	2001	HUN	First, may I congratulate Mr. Han Seung-soo o...
7506	56	2001	KWT	On behalf of the State of Kuwait, it\ngives m...

7507 rows x 4 columns

Fig 10.1 Screenshot of Dataset

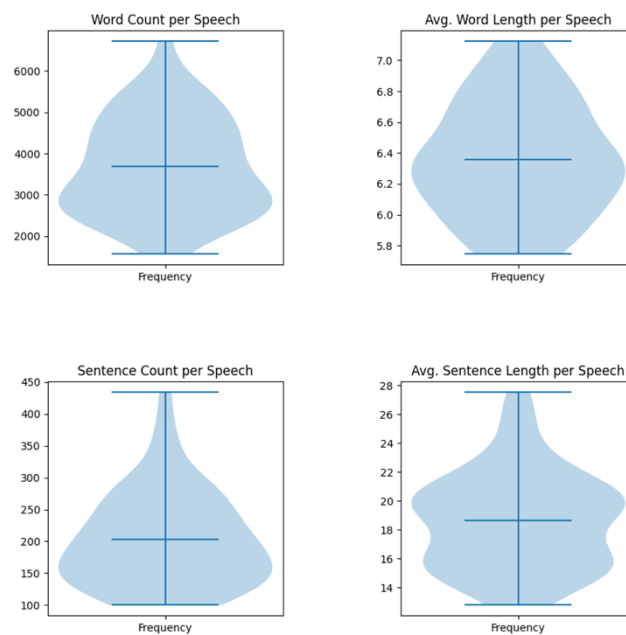


Fig 10.2 Vilon Plots for Count of words & Speech



Fig 10.5 Most used Bi-grams by the US at UN Debate sessions

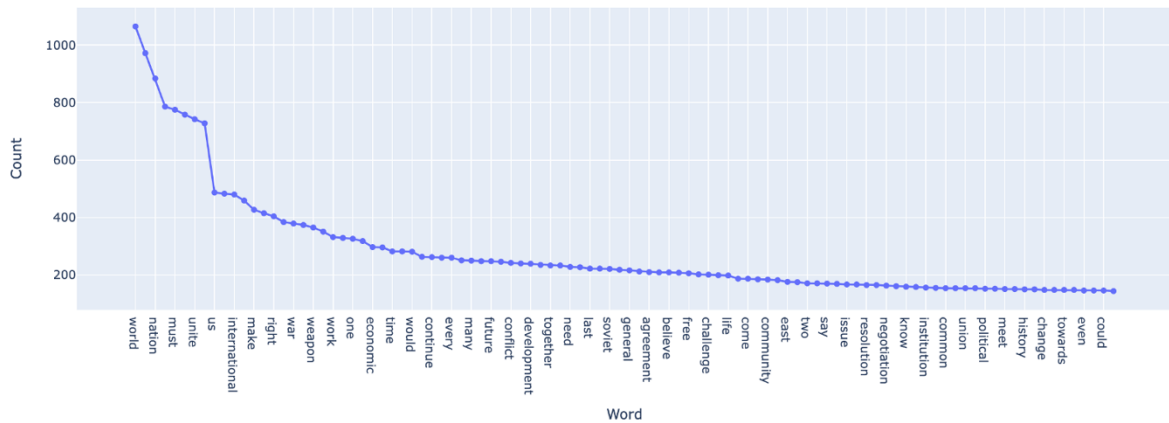


Fig 10.6 Frequency of Words Used Over Years

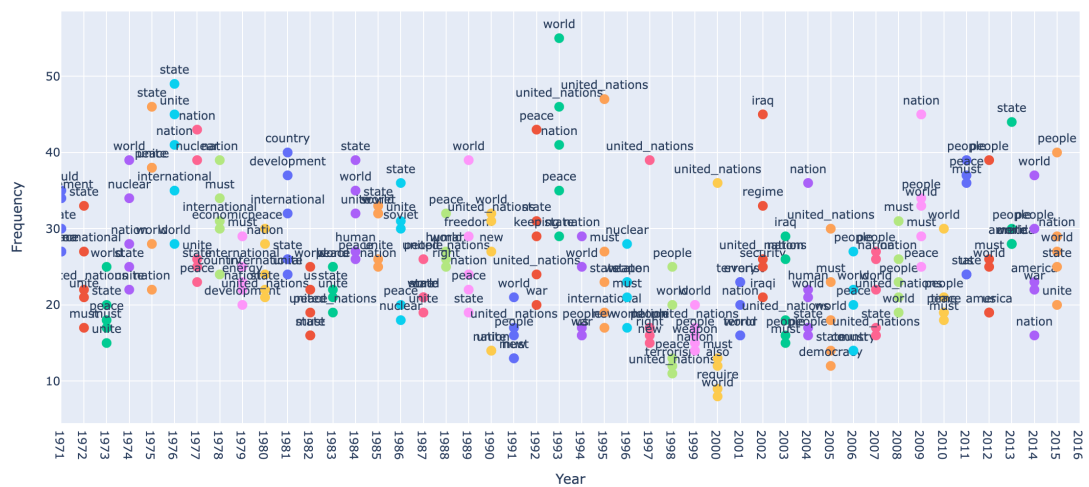


Fig 10.7 Top 5 Words Used Over the Years

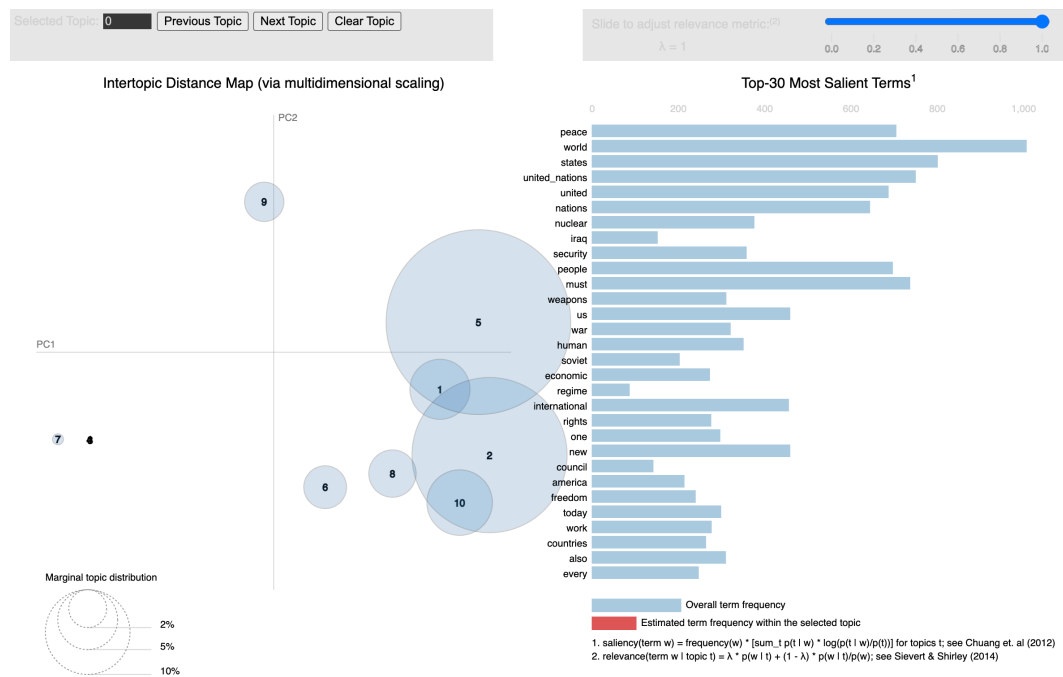


Fig 10.8 Topic Modeling for the Groups created

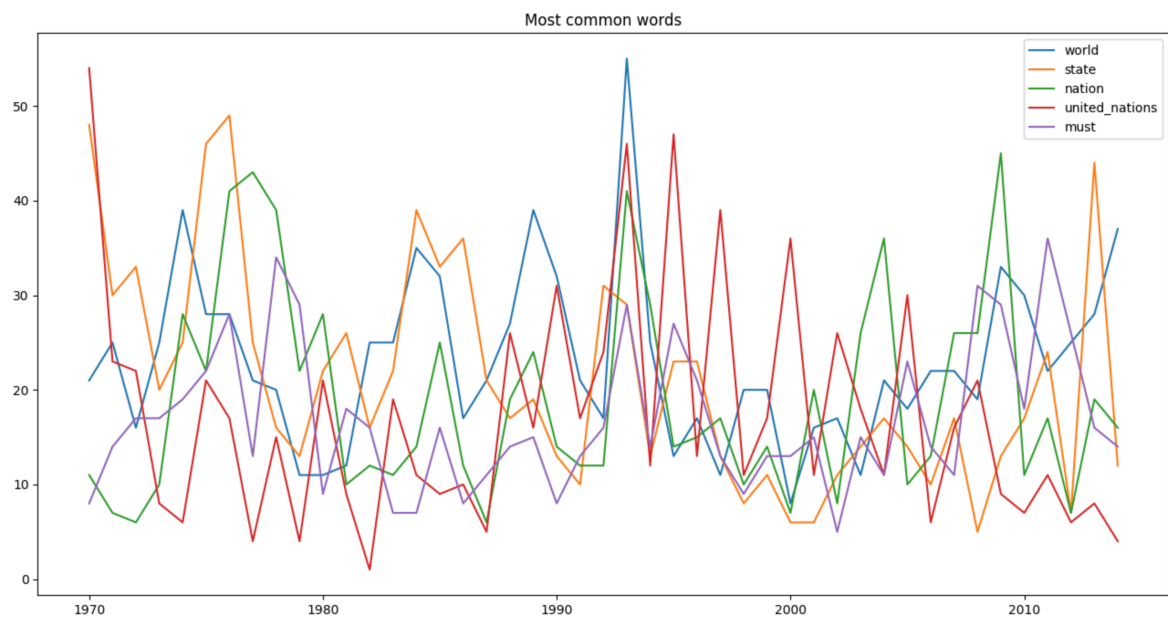


Fig 10.9 Most common words used by the US

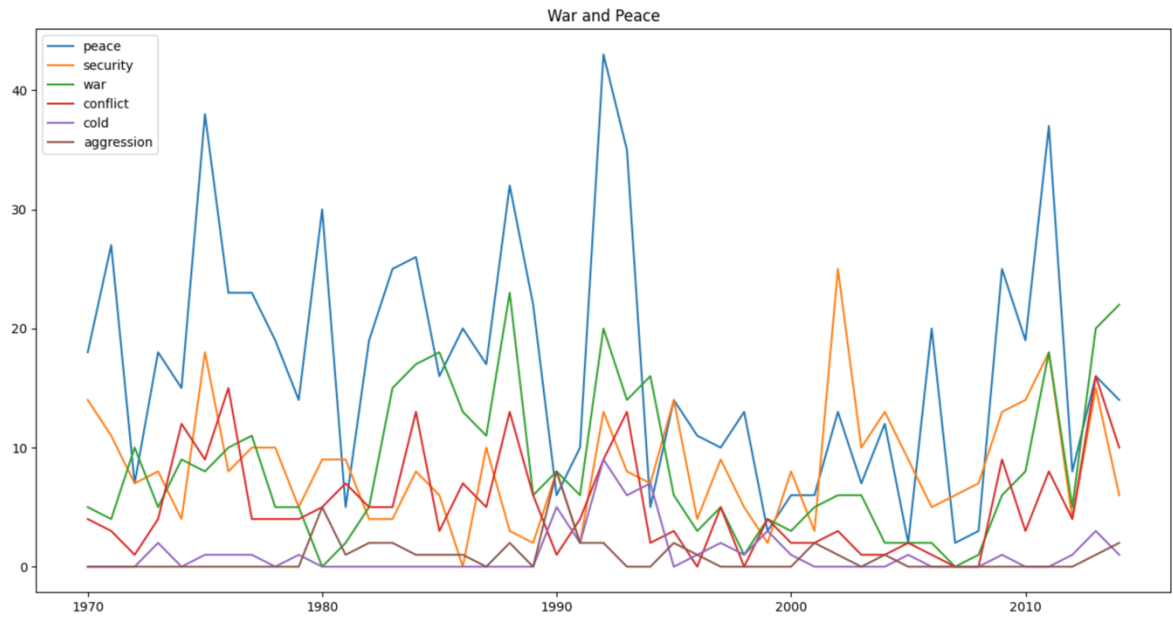


Fig 10.10 Most common words used by the US about War and Peace

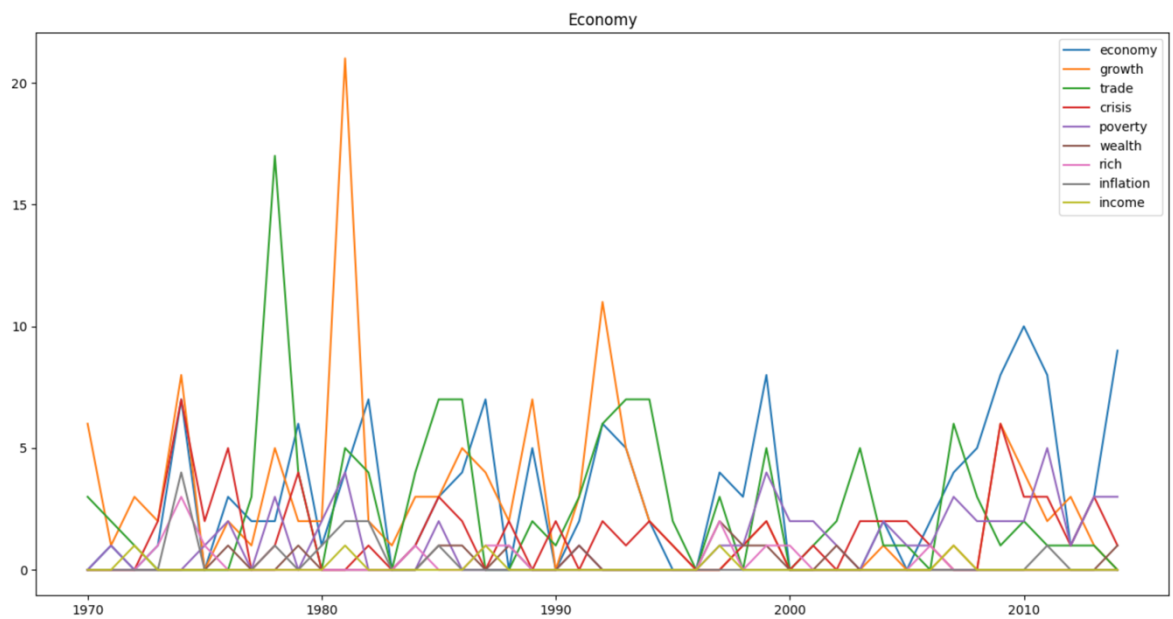


Fig 10.11 Most common words used by the US about Economy

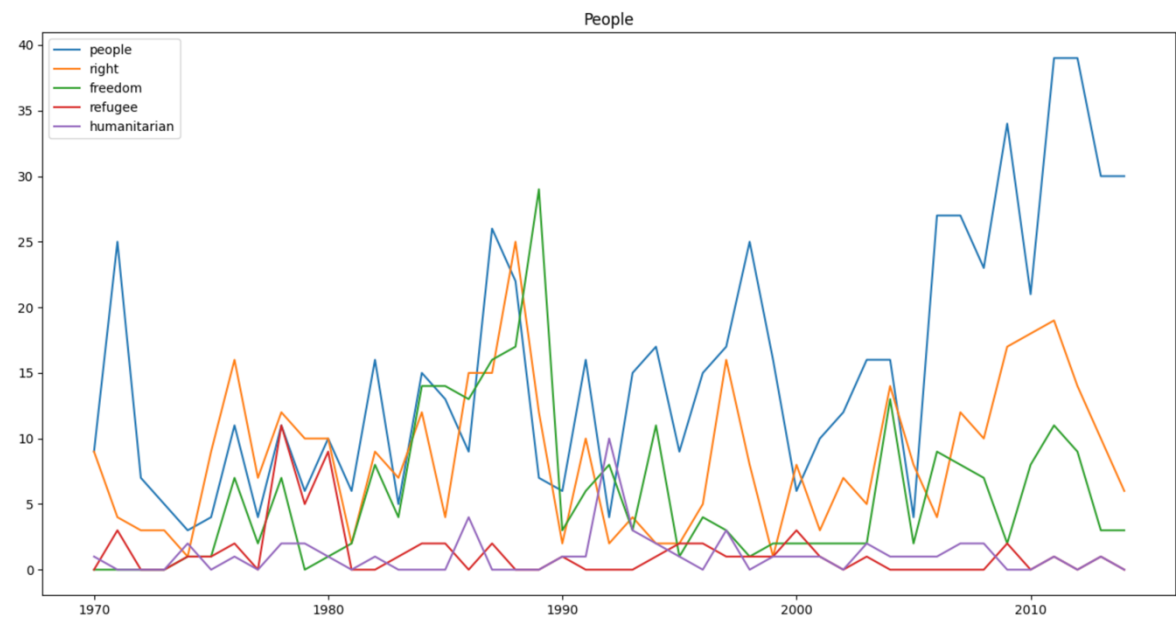


Fig 10.12 Most common words used by the US about People

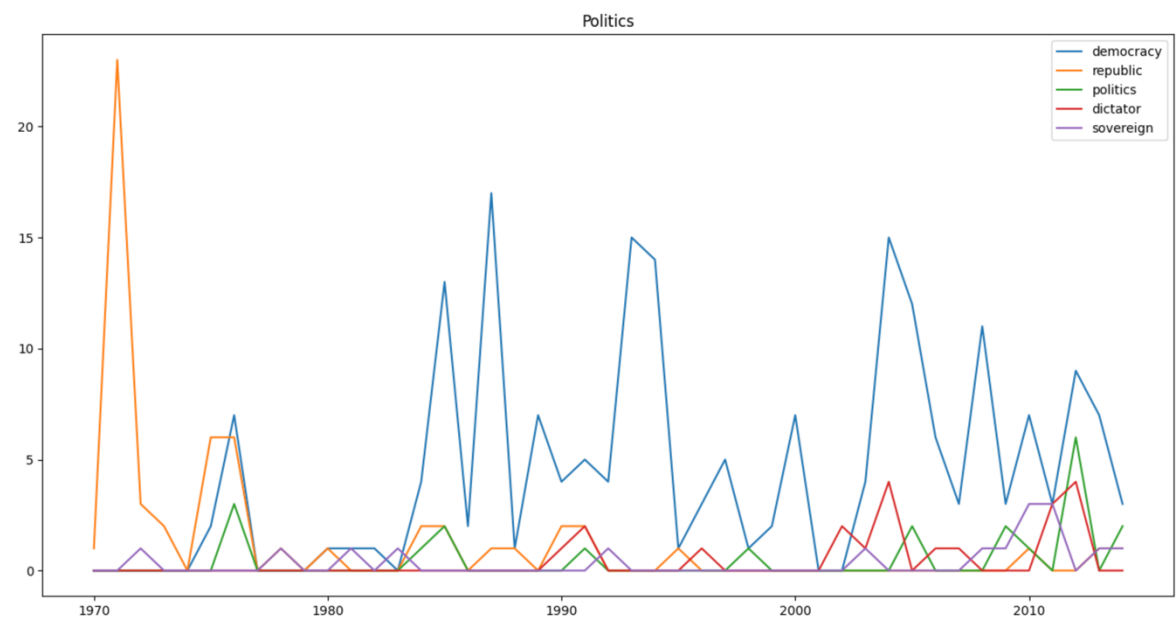


Fig 10.13 Most common words used by the US about Politics

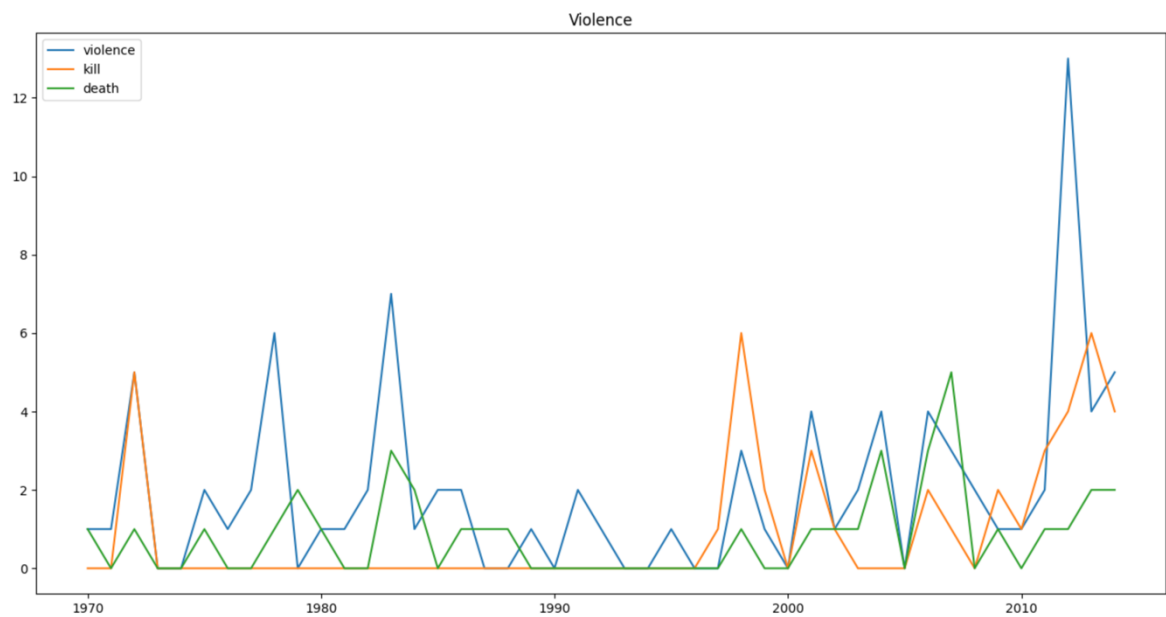


Fig 10.14 Most common words used by the US about Violence

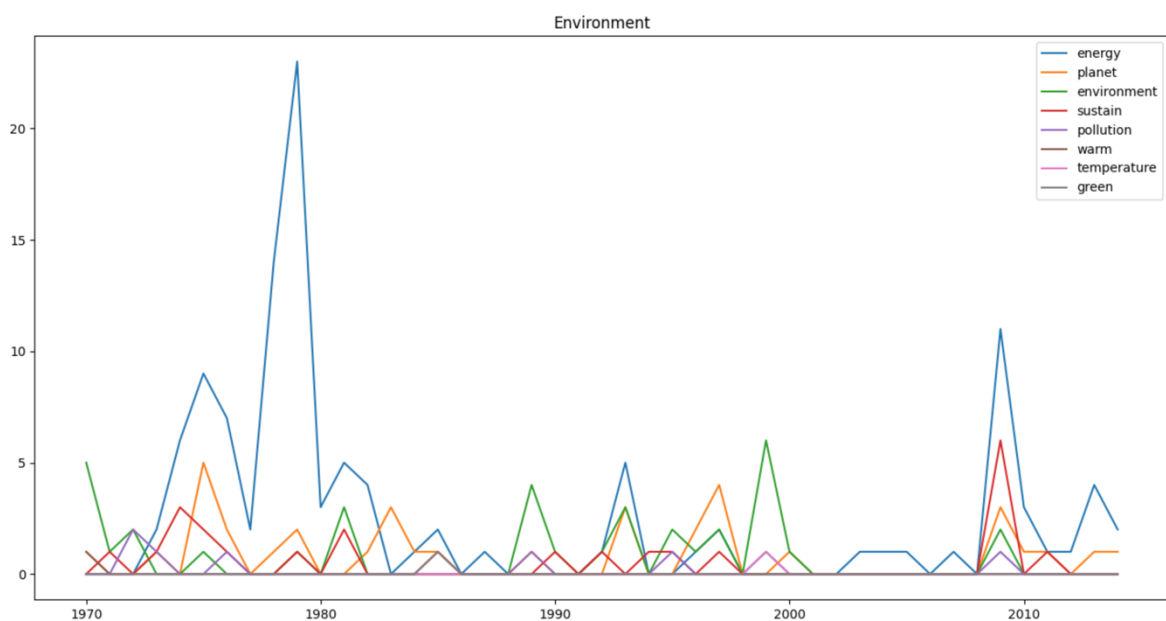


Fig 10.15 Most common words used by the US about Environment

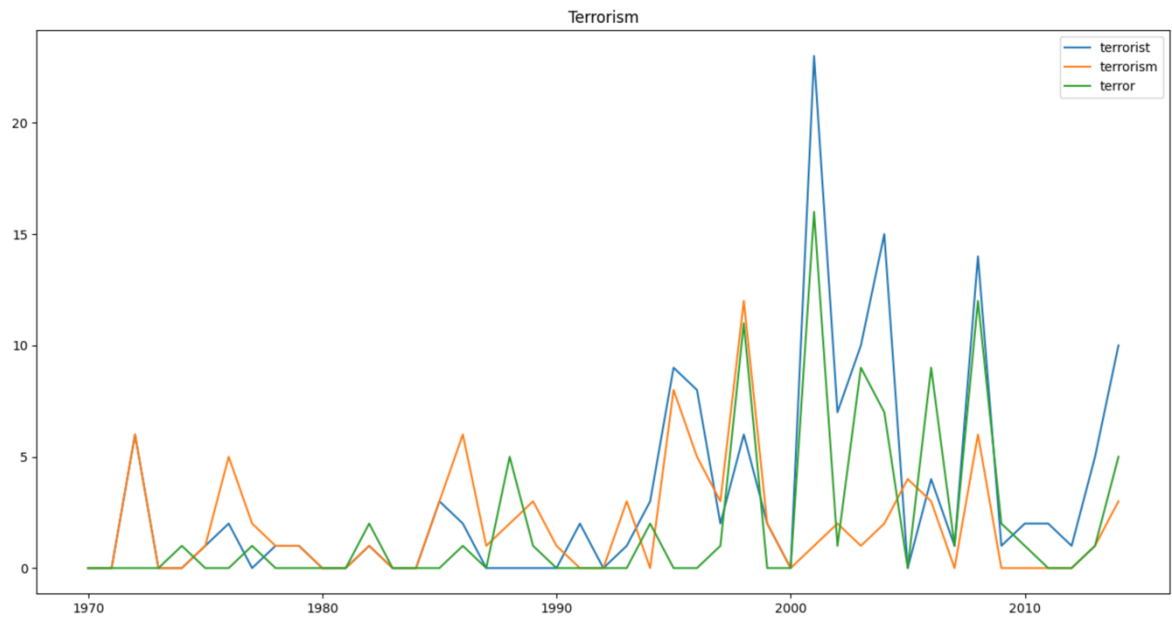


Fig 10.16 Most common words used by the US about Terrosim

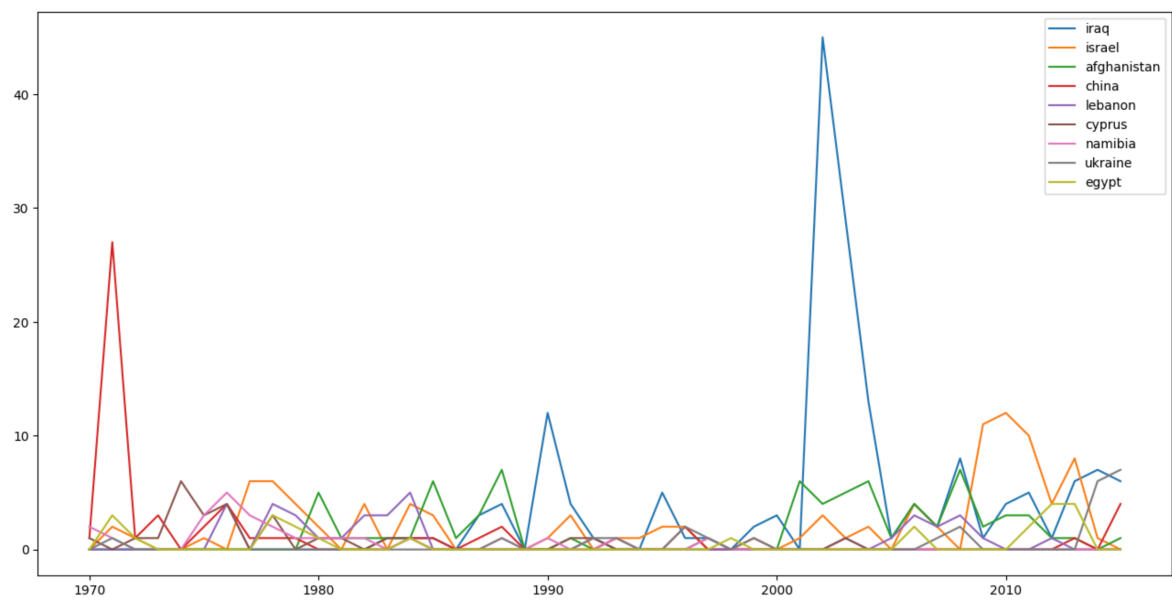


Fig 10.17 Most common Countires mentioned by the US

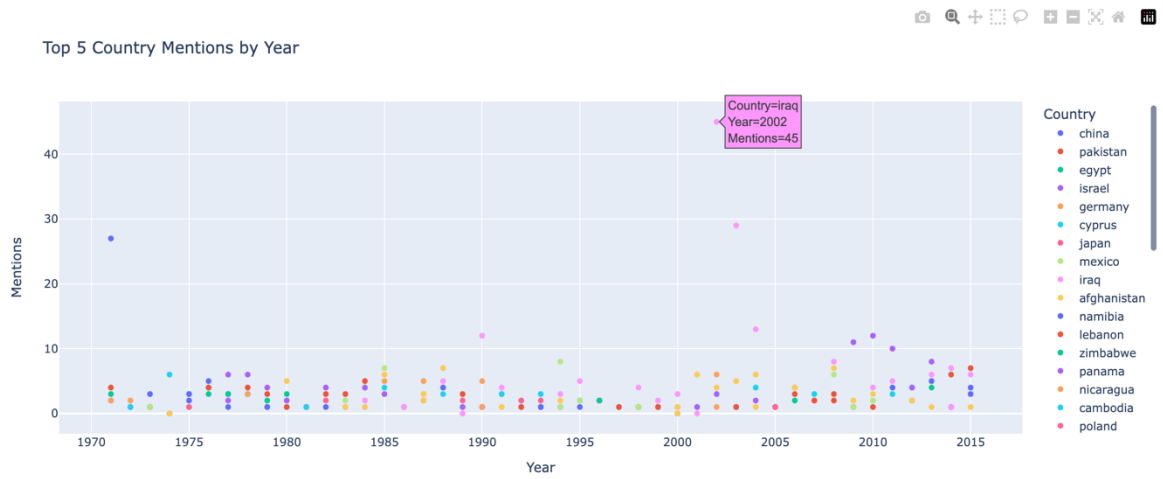


Fig 10.18 Most common Countires mentioned by the US in Scatter plot

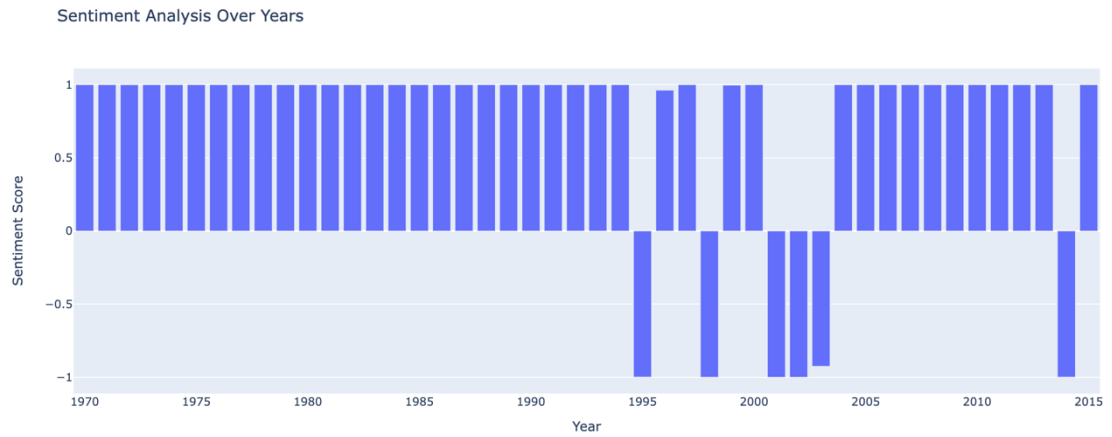


Fig 10.19 Sentiment analysis Score over the years

Polarity of Speeches by USA

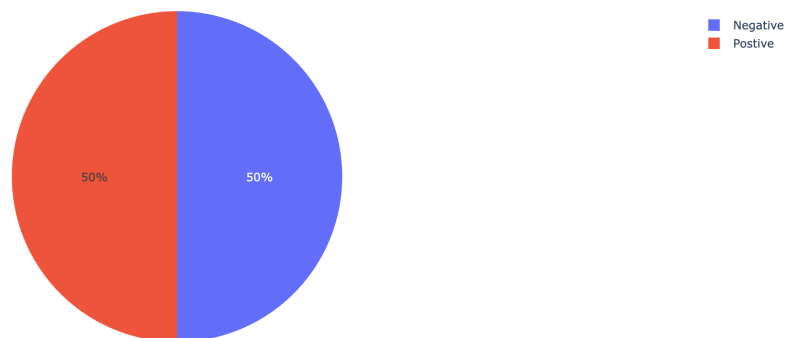


Fig 10.20 Polarity of Speeches by US

usa_mentioned		country
	691	cuba
	459	albania
	374	iraq
	369	ruissia
	368	libya
	367	panama
	353	nicaragua
	292	north korea
	280	belarus
	242	ukraine
	236	syria

Fig 10.21 Most Countires mentioned US

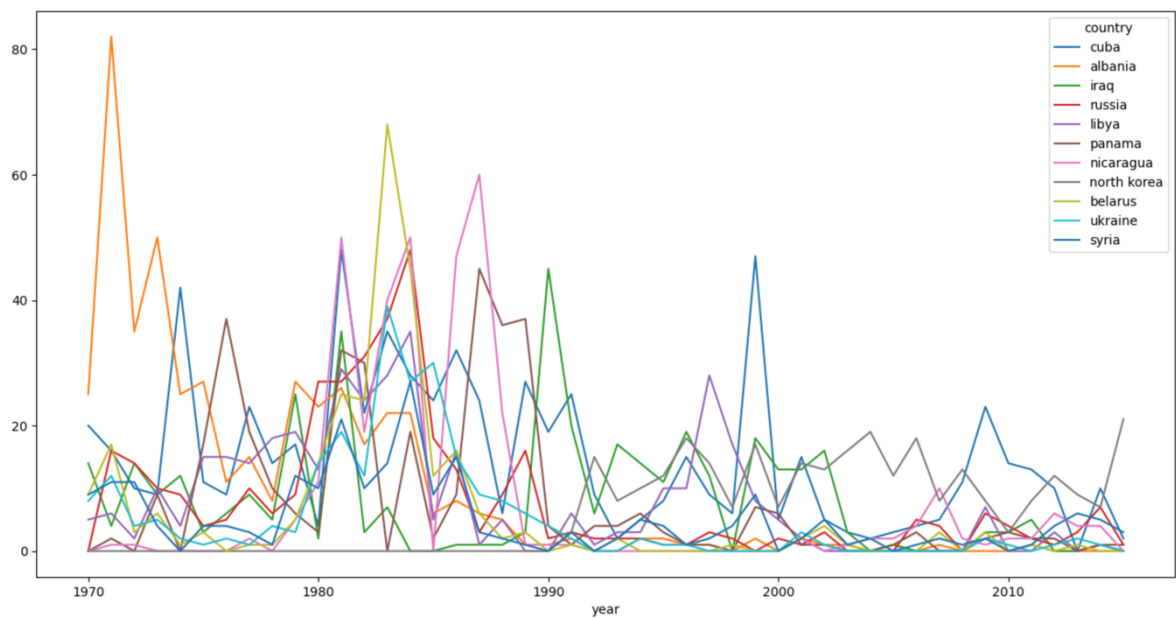


Fig 10.22 Most Countires mentioned US in a graph

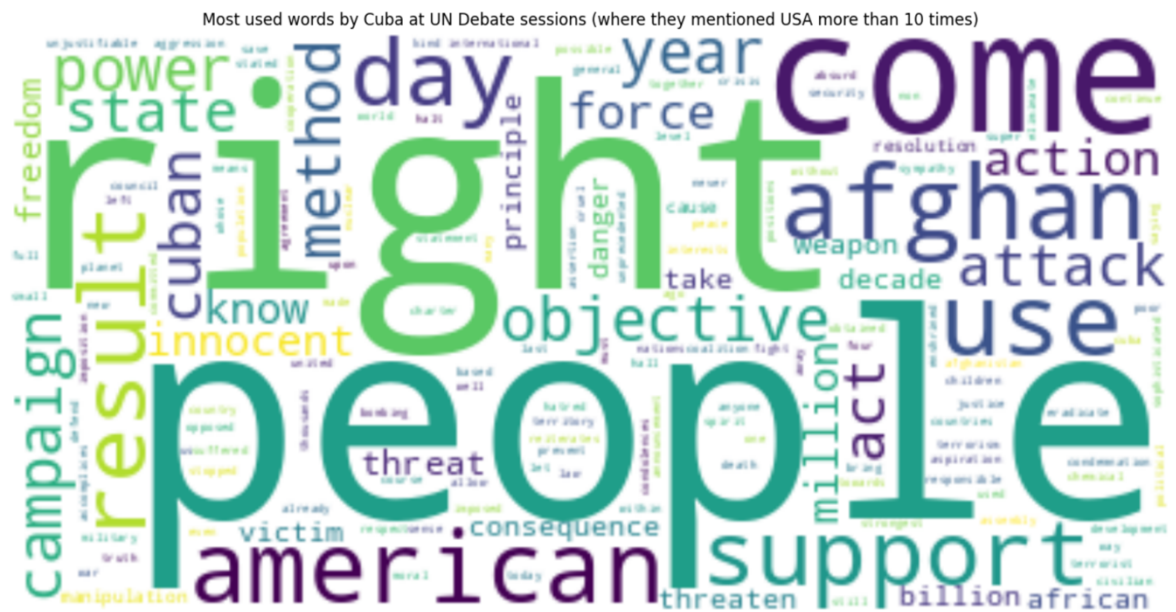


Fig 10.23 Most used words by Cuba at UN Debate(Where they mentioned USA more than 10 times)

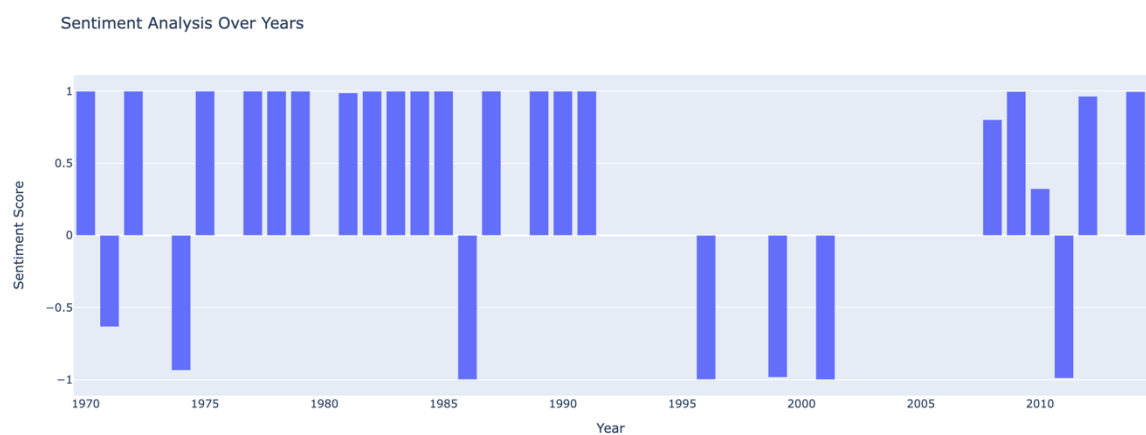


Fig 10.24 Sentiment Score of Cuba over the Years

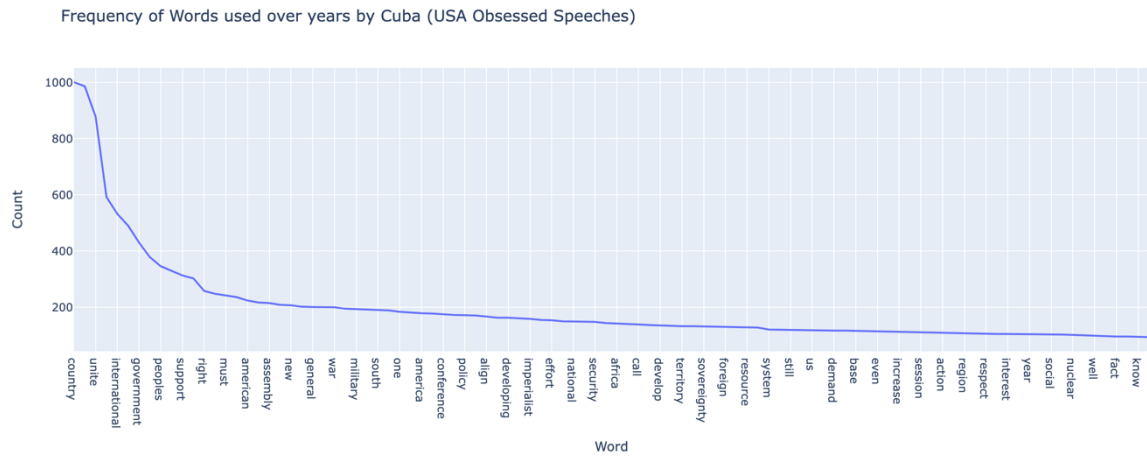


Fig 10.25 Frequency of Words used over years by cuba (USA Obsessed Speeches)

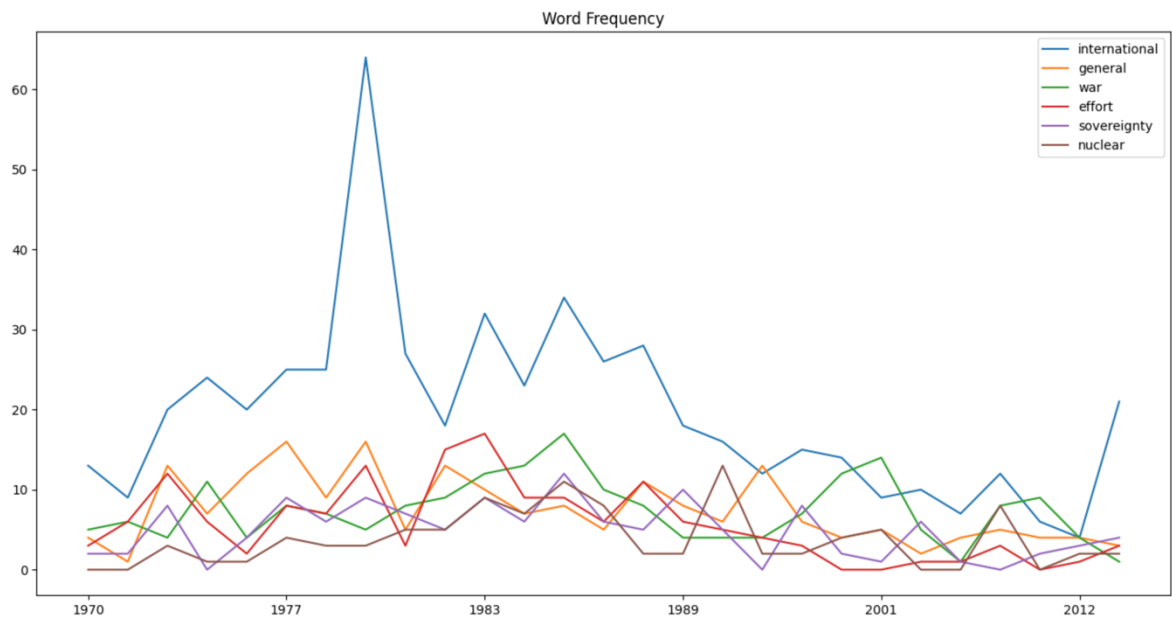


Fig 10.26 Word Frequency of cuba

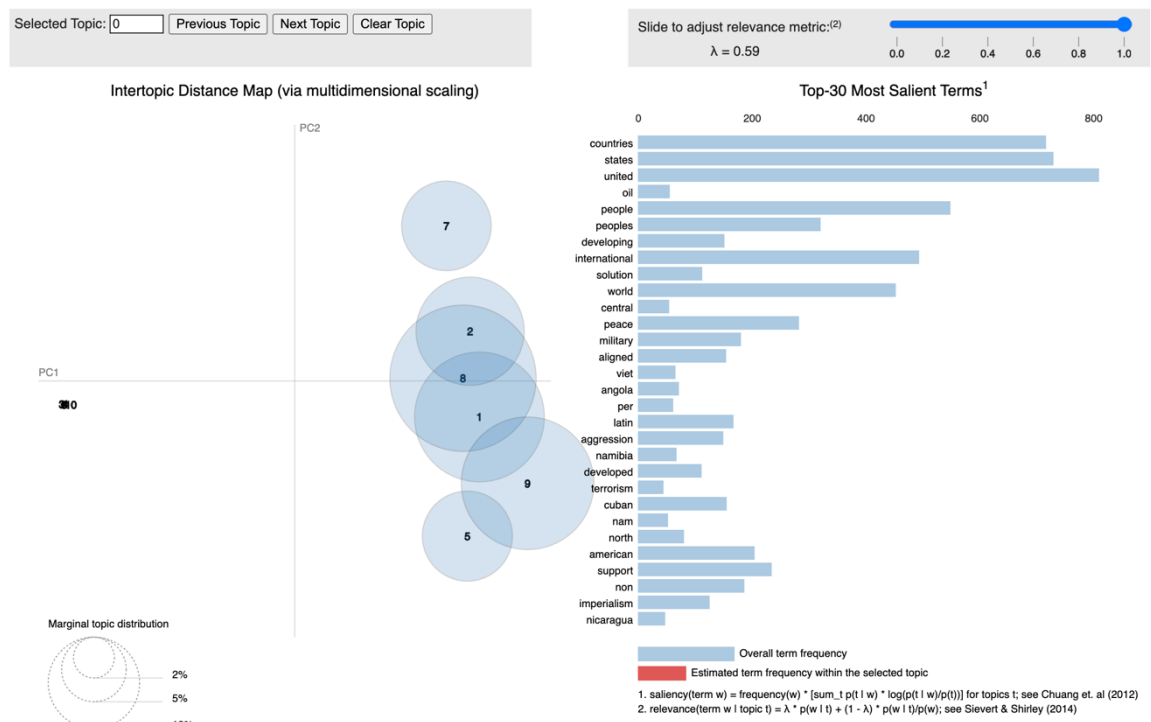


Fig 10.27 Topic Modeling of Cuba Speech

11. REFERENCES

Journal References:

- [1] *Dost-Mental Health Assistant Chatbot*, Nayar, A. M., Attar, Z., Kachwala, S., Biswas, T., & Wagh, S. K. (2022, December). *Dost-mental health assistant chatbot*. In *2022 5th International Conference on Advances in Science and Technology (ICAST)* (pp. 252-257). IEEE.
- [2] *Supervised Machine Learning Chatbots for Perinatal Mental Healthcare*, Wang, R., Wang, J., Liao, Y., & Wang, J. (2020, December). *Supervised machine learning chatbots for perinatal mental healthcare*. In *2020 International Conference on Intelligent Computing and Human-Computer Interaction (ICHCI)* (pp. 378-383). IEEE.
- [3] *MediBot: Healthcare Assistant on Mental Health and Well Being*, Kavyashree, N., & Usha, J. (2023, November). *MediBot: Healthcare Assistant on Mental Health and Well Being*. In *2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)* (pp. 1-5). IEEE.
- [4] *Carebot: A Mental Health ChatBot*, Crasto, R., Dias, L., Miranda, D., & Kayande, D. (2021, May). *Carebot: A mental health chatbot*. In *2021 2nd international conference for emerging technology (INCET)* (pp. 1-5). IEEE.
- [5] *Chatbot for Mental health support using NLP*, Gupta, V., Joshi, V., Jain, A., & Garg, I. (2023, May). *Chatbot for Mental health support using NLP*. In *2023 4th International Conference for Emerging Technology (INCET)* (pp. 1-6). IEEE.
- [6] *Mobile Application for Mental Health Using Machine Learning*, Mendis, E. S., Kasthuriarachchi, L. W., Samarasingha, H. P. K. L., Kasthuriarachchi, S., & Rajapaksa, S. (2022, December). *Mobile Application for Mental Health Using Machine Learning*. In *2022 4th International Conference on Advancements in Computing (ICAC)* (pp. 387-392). IEEE.
- [7] *Artificial Intelligence Powered Chatbot for Mental Healthcare based on Sentiment Analysis*, Mehta, A., Virkar, S., Khatri, J., Thakur, R., & Dalvi, A. (2022, December). *Artificial intelligence powered chatbot for mental healthcare based on sentiment analysis*. In *2022 5th International Conference on Advances in Science and Technology (ICAST)* (pp. 185-189). IEEE.
- [8] *Security and Privacy Perceptions of Mental Health Chatbots*, Chametka, P., Maqsood, S., & Chiasson, S. (2023, August). *Security and Privacy Perceptions of Mental Health Chatbots*. In *2023 20th Annual International Conference on Privacy, Security and Trust (PST)* (pp. 1-7). IEEE.

- [9] *The Use of AI Chatbots in Mental Healthcare for University Students in Thailand: A Case Study*, Mahmud, T. R., & Porntrakoon, P. (2023, August). *The Use of AI Chatbots in Mental Healthcare for University Students in Thailand: A Case Study*. In *2023 7th International Conference on Business and Information Management (ICBIM)* (pp. 48-53). IEEE.
- [10] *EmpathAI: Your Personal Emotional Support Chatbot*, Subba, N. D., & Dorji, T. (2023, August). *EmpathAI: Your Personal Emotional Support Chatbot*. In *2023 9th International Conference on Smart Computing and Communications (ICSCC)* (pp. 535-540). IEEE.